Fundamentals of Security in SAP BTP

15 90 20,809

Introduction

This blog series is mainly targeted for developers and administrators. If you are someone who has gone through the plethora of tutorials, documentation and presentations on security topics in SAP BTP and still lacks the confidence to implement security for your application, you have come to the right place.

In this blog series, we will learn:

- How to protect an app in Cloud Foundry environment of SAP BTP from unauthorized access
- How to implement role-based features
- How SAP Identity Provider and Single-sign-on works

For ease of reading, I have split this in multiple blogs, which are:

1. Fundamentals of Security in BTP: **Introduction [current blog]**
2. Fundamentals of Security in BTP: **What is OAuth?**
3. Fundamentals of Security in BTP: Implement Authentication and Authorization in a Node.js App
4. Run Node.js Applications with Authentication Locally

For a comprehensive learning path, you may also check How to Become Expert in SAP BTP Security– A Complete Learning Journey

Note: If you are new to SAP BTP and looking for a simple explanation of what it is and what problem it solves, see Explaining SAP Business Technology Platform (SAP BTP) to a Beginner

Is security in SAP BTP Complicated?

In my opinion, security in SAP BTP is a simple topic. All we need is a clear understanding of all the basic concepts.

Let's be honest that most of the developers don't design the application keeping security in mind at first place. We always try to prove the feasibility of use-case first. Once we succeed, then only we think about securing the application. One of the reasons, why people get on back foot when it comes to security implementation.

In SAP BTP, it's super easy to develop a full-stack business application using sophisticated frameworks like *SAP Cloud Application Programming Model*, *SAP Cloud SDK* and many other out-of-the-box cloud services. With little bit effort on

understanding the core concepts, you can also solve all the pieces of puzzle in security.

In this blog series, we will make it extremely simple. First, we will understand the basic concepts:

- Identity Provider (IdP)
- XSUAA
- OAuth
- Application Router
- Authentication and Authorization Implementation etc.

Further, to get a hands-on experience, we will

1. First deploy an Unsecured Hello World Node.js application in BTP Cloud Foundry.
2. Then step-by-step we will implement authentication and authorization.
3. We will also touch upon Identity Provider to be used for single-sign-on.

Note: This blog series is focused on SAP BTP, **Cloud Foundry environment**. I may skip mentioning Cloud Foundry every time.

Are there any prerequisites for this blog?

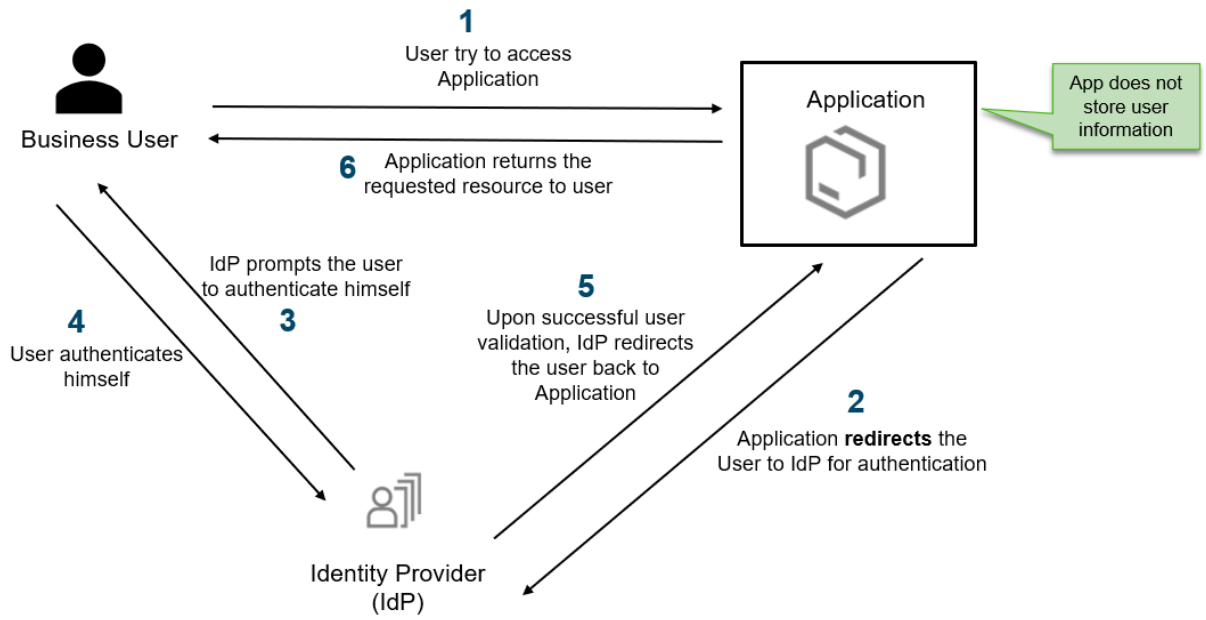
You should have basic idea of SAP BTP, BTP Cockpit and basic Node.js skill to understand these series. Even if you are new to Node.js, you should be managed to go through it.

Let’s start with the basic concepts.

Identity Provider (IdP)

Applications in SAP BTP does not store user information. Instead, the applications redirect the authentication to an Identity Provider. This concept makes it possible to decouple and centralize authentication functionality.

Below image shows a very high-level architecture of a typical Identity Provider.



In SAP BTP, there are 2 options for Identity Provider – **SAP ID Service** and **SAP Cloud Identity Authentication service (IAS)**.

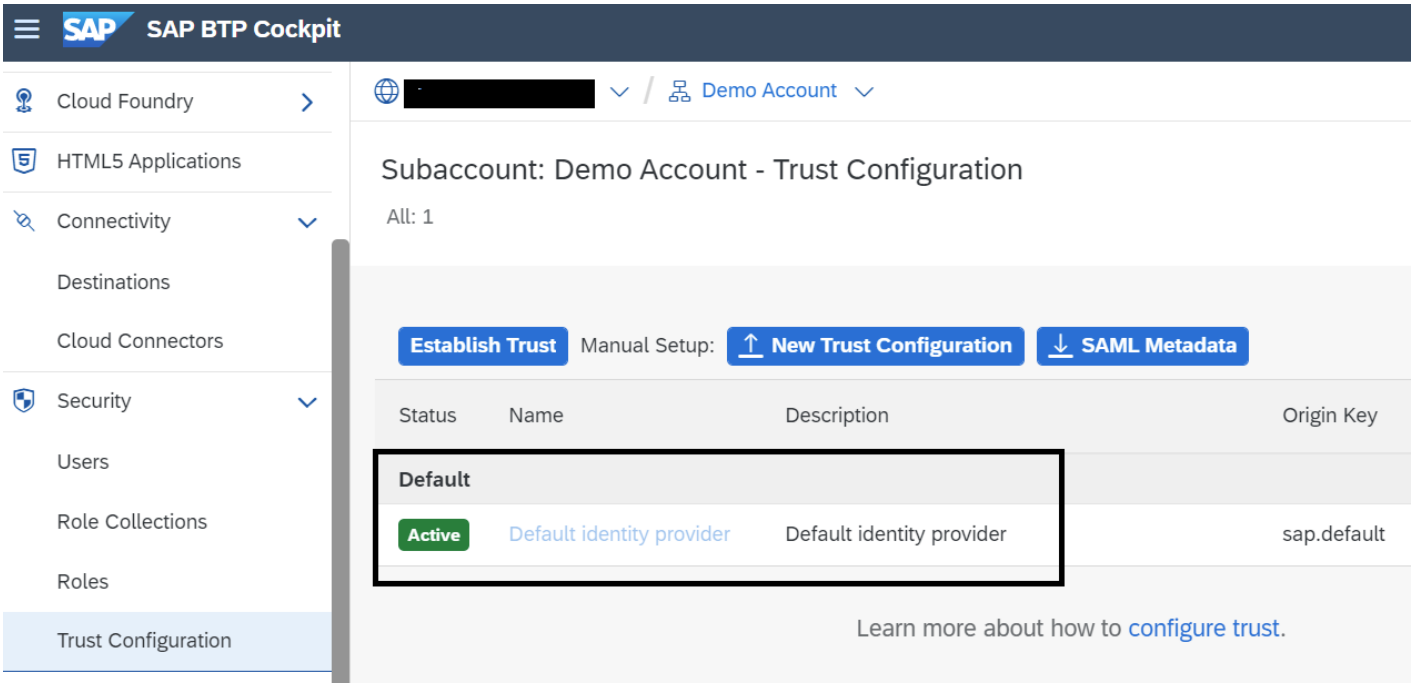
SAP ID Service

SAP ID Service is the **default** identity provider in SAP BTP. It is a **pre-configured**, standard SAP public IdP (account.sap.com) that is shared by all customers.

Few important points about SAP ID Service:

- Trust to SAP ID service is pre-configured in all BTP subaccounts.
- SAP ID Service is managed by SAP.
- SAP ID service manages the users of official SAP sites, including the SAP developer and partner community. It is the place where the S-Users, P-Users, and D-Users are managed.

You can view the SAP ID service pre-configured in BTP subaccounts in the cockpit, as shown below.

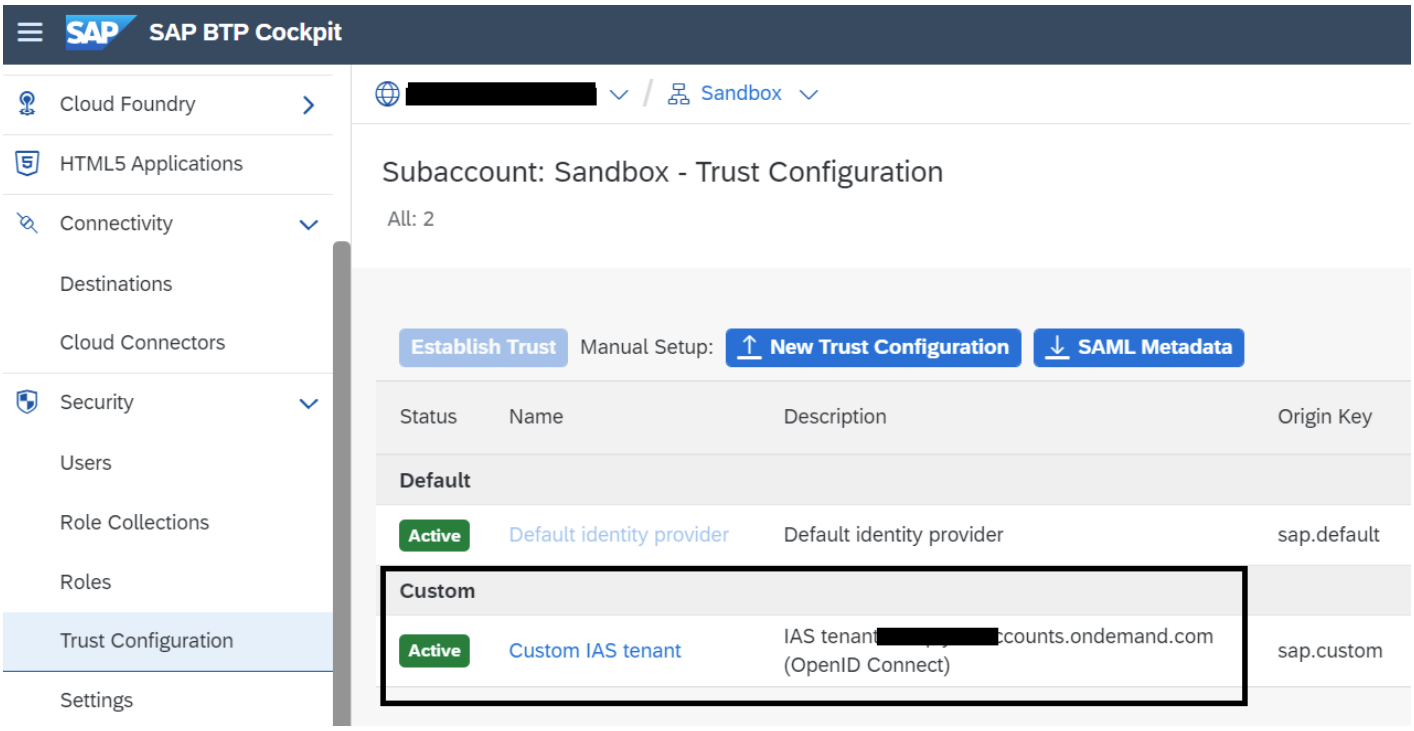


SAP Cloud Identity Authentication service (IAS)

For many customers, business users might be stored in corporate identity providers. SAP recommends using **SAP Cloud Identity Services – Identity Authentication Service (IAS)** as a hub.

We can connect IAS as a **single custom identity provider** to SAP BTP. Further use IAS to integrate with corporate identity providers.

IAS can be configured in SAP BTP cockpit, in the Trust Configuration section, as shown below.



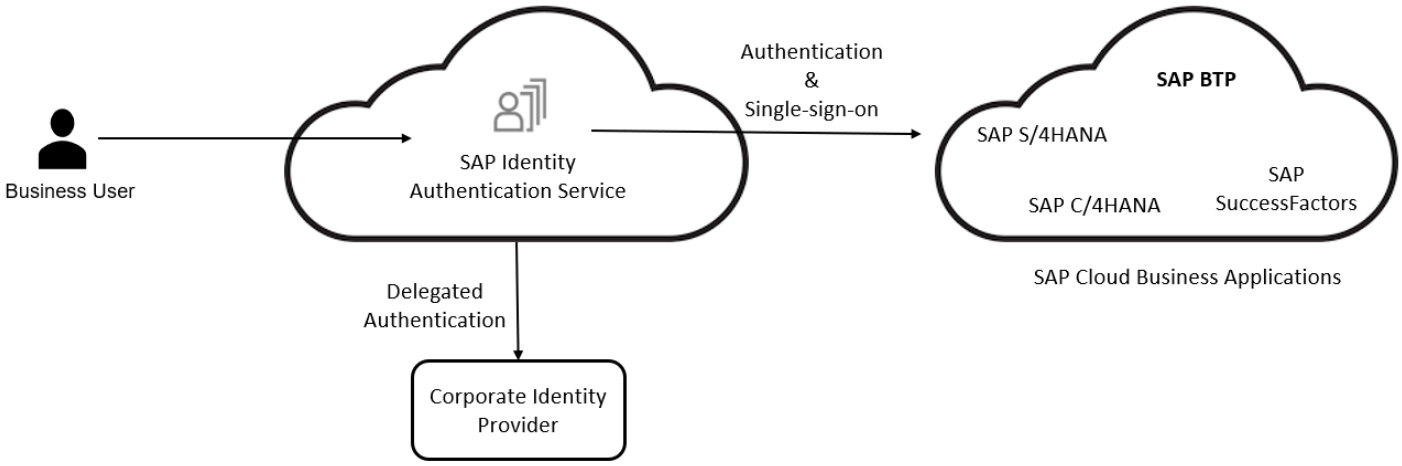
To know more on establishing trust between SAP BTP and IAS, you may refer to the help document.

Why do we really need SAP Identity Authentication Service (IAS)?

Most customers already have huge on-premises or cloud ecosystem. Their business user data is already available in their corporate identity provider.

When these customers build applications on BTP, an important question comes up – "*How can employees authenticate to the applications with **known credentials**?*"

In simple words, customer needs to provide **single sign-on for their custom solution on BTP, SAP S/4HANA Cloud, SAP SuccessFactors and other SAP solutions.** The answer to this is SAP Identity Authentication Service.



As shown in the above image, IAS can either act as an IdP itself or delegate the authentication to a corporate identity provider. IAS acts a central hub to provide single-sign-on to all SAP cloud applications as well as BTP applications.

SAP Authorization and Trust Management Service (XSUAA)

The XSUAA is one of the most important components involved in security. Let's get hold of this.

What is XSUAA?

SAP XSUAA is an **internal development of SAP**.

In Cloud Foundry, there is an open-source component called UAA. UAA is an OAuth provider which takes care of authentication and authorization. SAP took the base of UAA and extended it with SAP specific features to be used in SAP BTP.

Technically XSUAA is an **OAuth server** and uses JWT tokens.

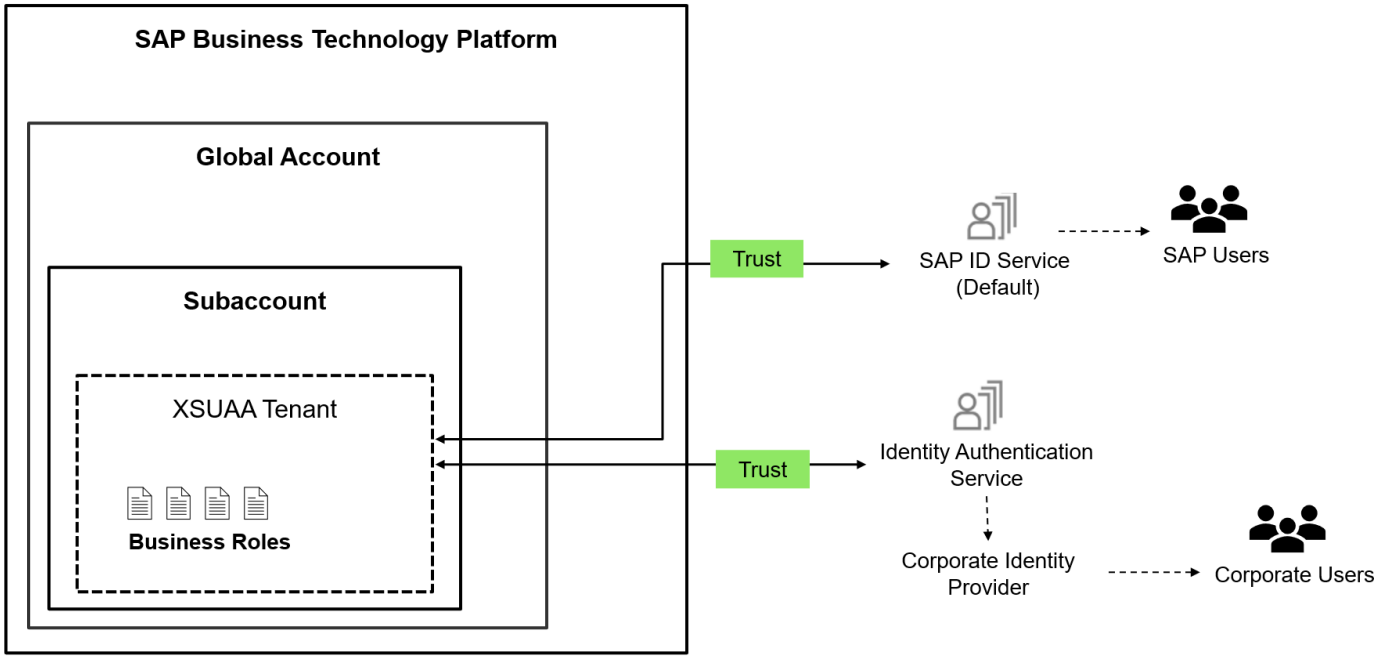
If you are new to OAuth, refer to this blog to know more about it:

- What is OAuth and how does it work?

What problem does XSUAA solve?

XSUAA takes care of authentication and authorization in SAP BTP, Cloud Foundry.

Below image shows very high-level architecture of XSUAA. It's important to note that, XSUAA does NOT store users data. This is why the XSUAA needs to trust an external Identity Provider (IdP). It can **establish trust** either with **SAP ID Service** or a **Corporate Identity Provider via SAP Identity Authentication Service (IAS)**.



Application Router

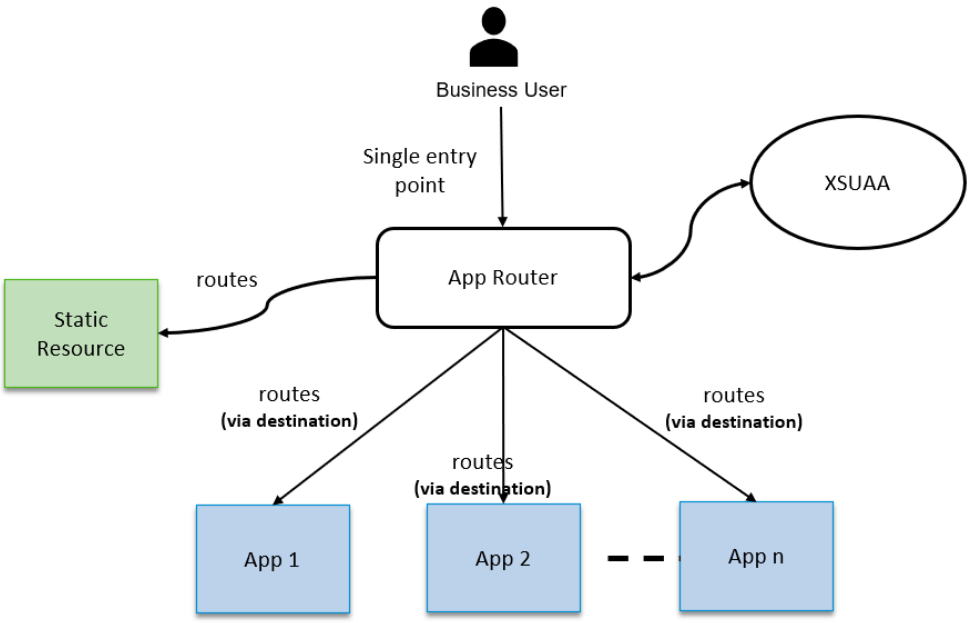
When a business application consists of several different apps (microservices), the application router is used to provide a **single-entry-point** to the business application.

Technically, Application Router is a Node.js App, available in public in NPM.

What is the purpose of Application Router?

App Router is used to:

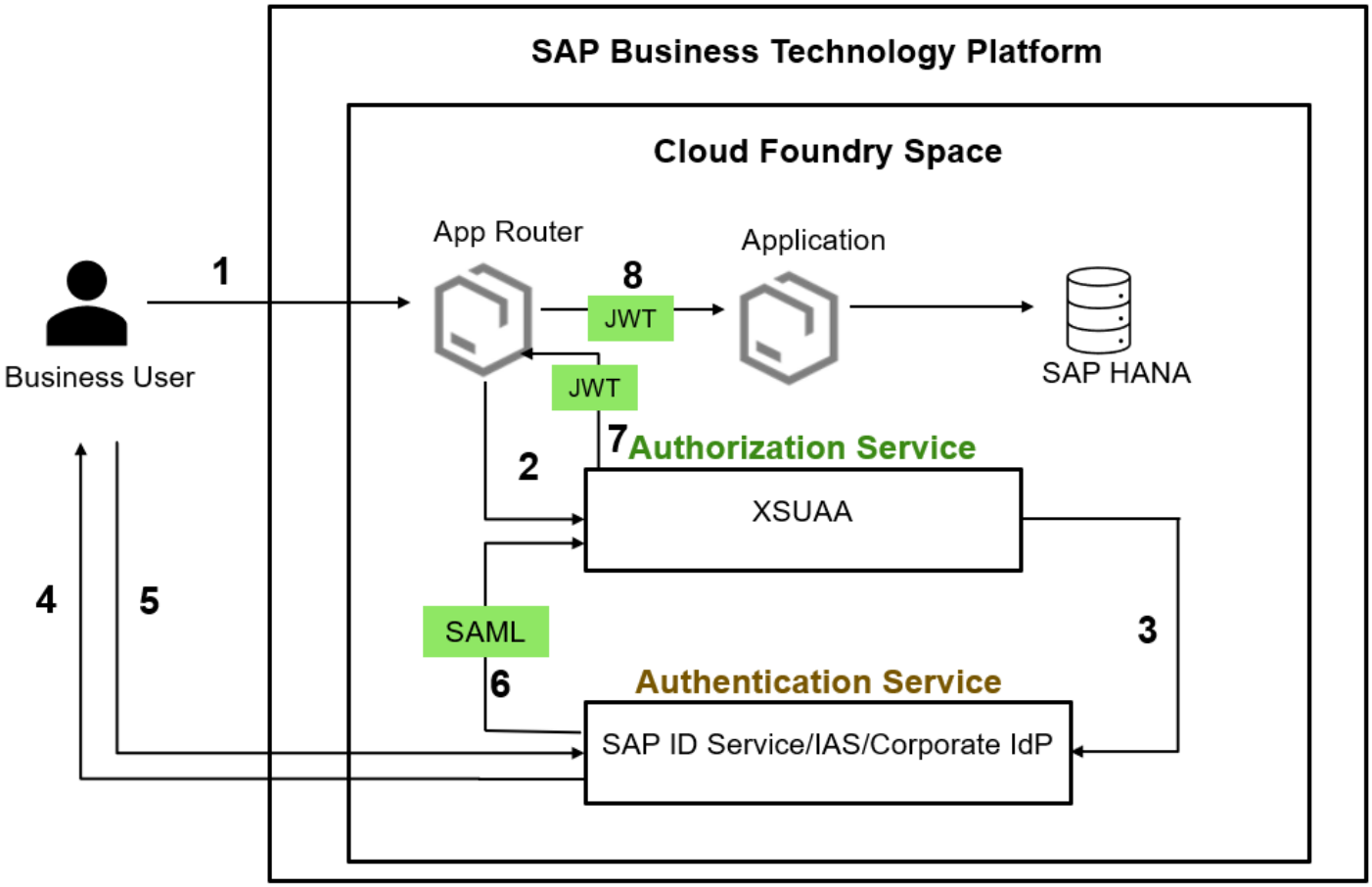
- Serve static content
- Authenticate users
- Dispatch request to backend applications(microservices)



Note that **App Router delegates the authentication responsibility to XSUAA**.

Call Flow – Putting all the pieces together

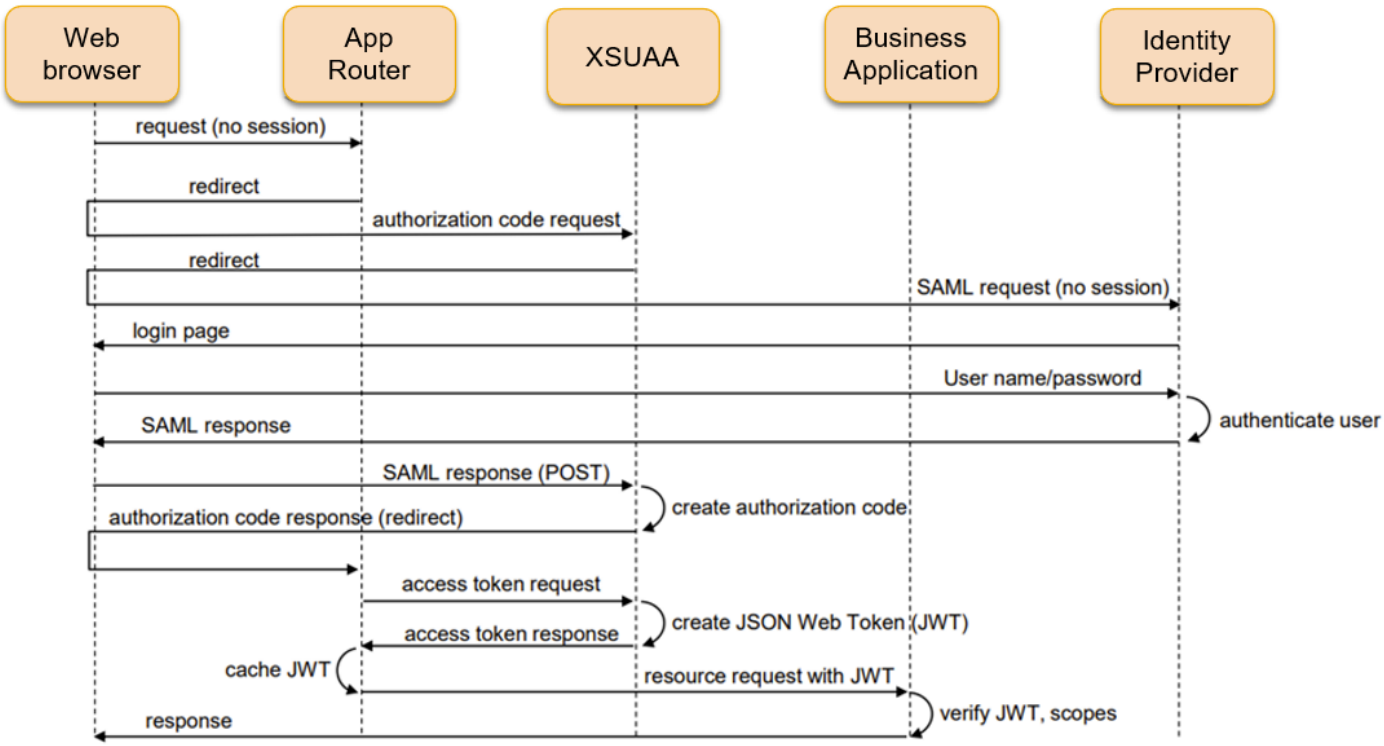
Now, since we have got clear idea on individual components, let's understand how these all work together.



The above diagram showcases the call flow. Let's break it down.

1. User request for the resource from Application. The **App Router takes incoming**.
2. Since user is not authenticated, **App Router initiates an OAuth2 flow with the XSUAA**.
3. **XSUAA forwards the request to Identity Provider** to enforce the business user to authenticate.
4. **IdP prompts the user to authenticate** himself. For Example, by entering username and password.
5. User authenticates himself.
6. If the authentication was successful, **Identity Provider sends a SAML token** to user (web browser). The web browser sends this new **SAML token** to the XSUAA for authentication.
7. XSUAA consider this request as authenticated and generates an OAuth Token which is technically a **JWT token**.
8. The App Router **enriches each subsequent request with the JWT**, before the request is routed to a dedicated application. The application verify the JWT token and send the requested resource to user.

The below image showcases the same thing as sequence diagram.



I hope you got the basic idea of Identity Provider, XSUAA and App Router.

Fundamentals of Security in SAP BTP – What is OAuth and how does it work?

This blog series is mainly targeted for developers and administrators. If you are someone who has gone through the plethora of tutorials, documentation, and presentations on security topics in SAP BTP and still lacks the confidence to implement security for your application, you have come to the right place.

In this blog series, we will learn:

- How to protect an app in SAP BTP, Cloud Foundry environment from unauthorized access
- How to implement role-based features
- How SAP Identity Provider and Single-sign-on works

For ease of reading, I have split this in multiple blogs, which are:

1. Fundamentals of Security in BTP: **Introduction**
2. Fundamentals of Security in BTP: **OAuth Concept** ***[current blog]***
3. Fundamentals of Security in BTP: Implement Authentication and Authorization in a Node.js App
4. Run Node.js Applications with Authentication Locally

What are we going to learn in this blog?

In this blog, we will focus on OAuth. We will learn what OAuth is, how does it work.

Why was OAuth introduced?

Let's first understand why OAuth was introduced, what problem did it solve?

We know that we should never ever share our passwords. But there are use cases which require us to authorize a website to use the service of another. For example, you may need to tell Facebook that it's ok for ESPN.com to access your profile or post updates to your timeline. Instead of asking for your Facebook password, ESPN can use a protocol called OAuth.

What is OAuth?

OAuth is an open standard for applications and websites to handle authorization.

OAuth **doesn't share password** data but instead **uses authorization tokens** to prove an identity between consumers and service providers. It is an authentication protocol that allows you to approve one application interacting with another on your behalf without giving away your password.

OAuth:

- **Avoids storing credentials** at the third-party location
- **Limits the access permissions** granted to third parties
- **Enables easy access right revocation** without the need to change credentials

In this way, OAuth mitigates some of the common concerns with authorization scenarios

Note that **OAuth is about authorization and not authentication**. Authorization is asking for permission to do stuff. Authentication is about proving you are the correct person because you know things.

SAML vs. OAuth

The Security Assertion Markup Language (SAML) is an **open standard based on XML,** which many enterprises use for **Single-Sign On (SSO)**.

SAML enables enterprises to monitor who has access to corporate resources.

There are many differences between SAML and OAuth.

SAML uses **XML** to pass messages, and OAuth uses **JSON**.

OAuth is much more lightweight and an **ideal fit for system-to-system communication.** While SAML is geared towards enterprise security.

The key differentiator is:

- **OAuth uses API calls** extensively, which is why mobile applications, modern web applications, game consoles, and Internet of Things (IoT) devices find OAuth a better experience for the user.
- SAML, on the other hand, uses a session cookie in a browser that allows a user to access certain web pages – great for short-lived work days, but not so great when have to log into your smart watch every day.

How does OAuth work?

Let's say you want ESPN to post data directly on your Facebook page.

There are 4 actors in the game.

Resource Server: The server hosting the protected resources (e. g. Facebook, Twitter)

Resource Owner: User who owns the data in the resource server. For example, a User is the Resource Owner of his Facebook profile.

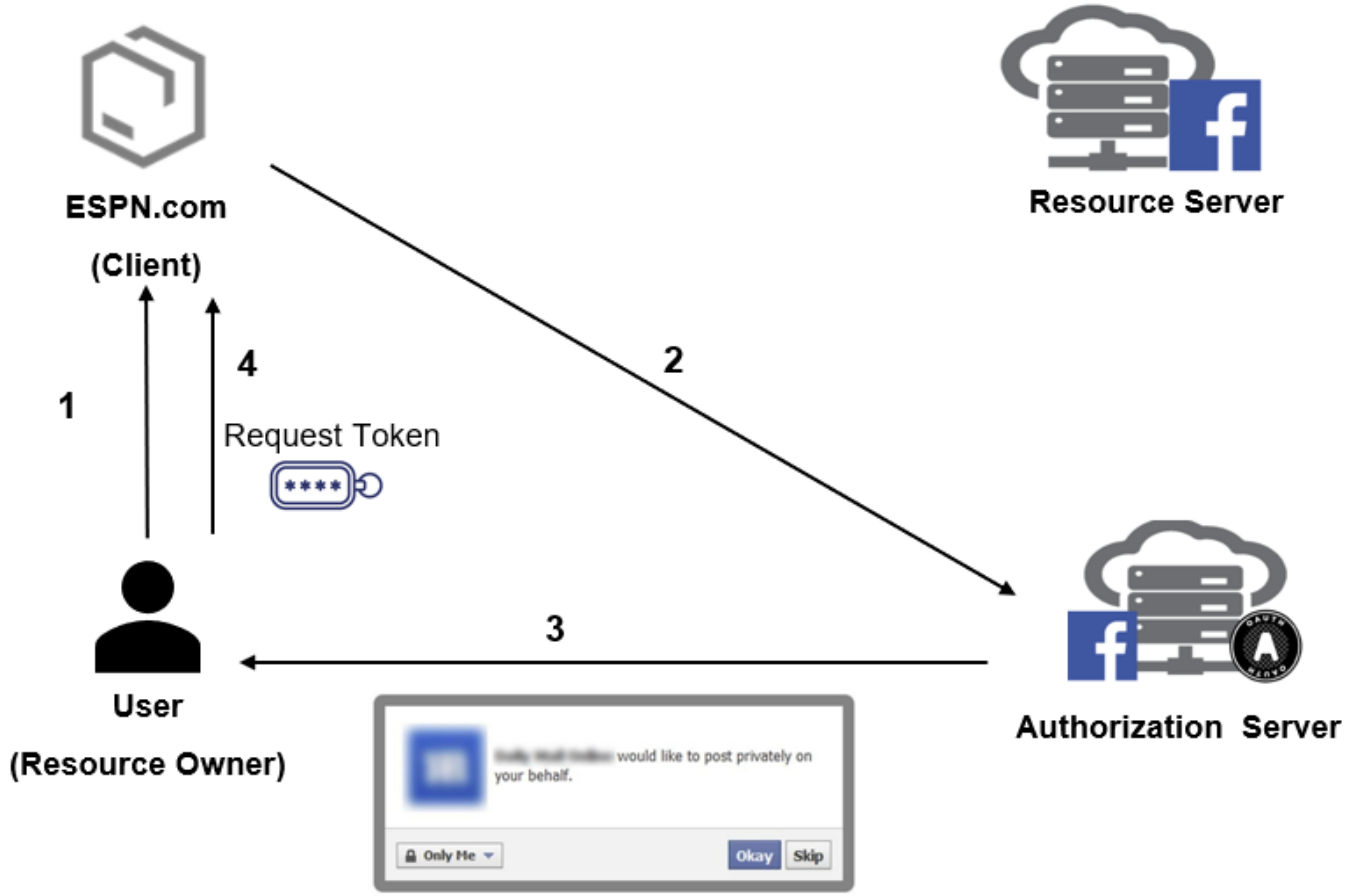
Client: The application that wants to access your data (e. g. ESPN.com)

Authorization Server: The main engine of OAuth. It allows Resource Owner to delegate his authorization to Client.

The entire flows work in 2 stages.

1. First User authorizes the ESPN – Front Channel Flow
2. ESPN Posts on User's Facebook Page – Back Channel

Front Channel Flow:



Step 1 – User Shows Intent to Client

User (Resource Owner): "Hey, ESPN, I would like you to be able to post directly to my Facebook page."

ESPN (Client): "Great! Let me go ask for permission."

Step 2 – The Client seeks Permission from Authorization Server

ESPN (Client): "Authorization Server, I have a user that would like me to post to his page. Can I have a Request Token?"

ESPN sends the request to Authorization Server with desired scopes. Scope is nothing but individual tasks (e.g. access friend list, post on page etc.)

Authorization Server: "Sure. But let me get a confirmation from the User."

Step 3 – Authorization Server asks for confirmation from user

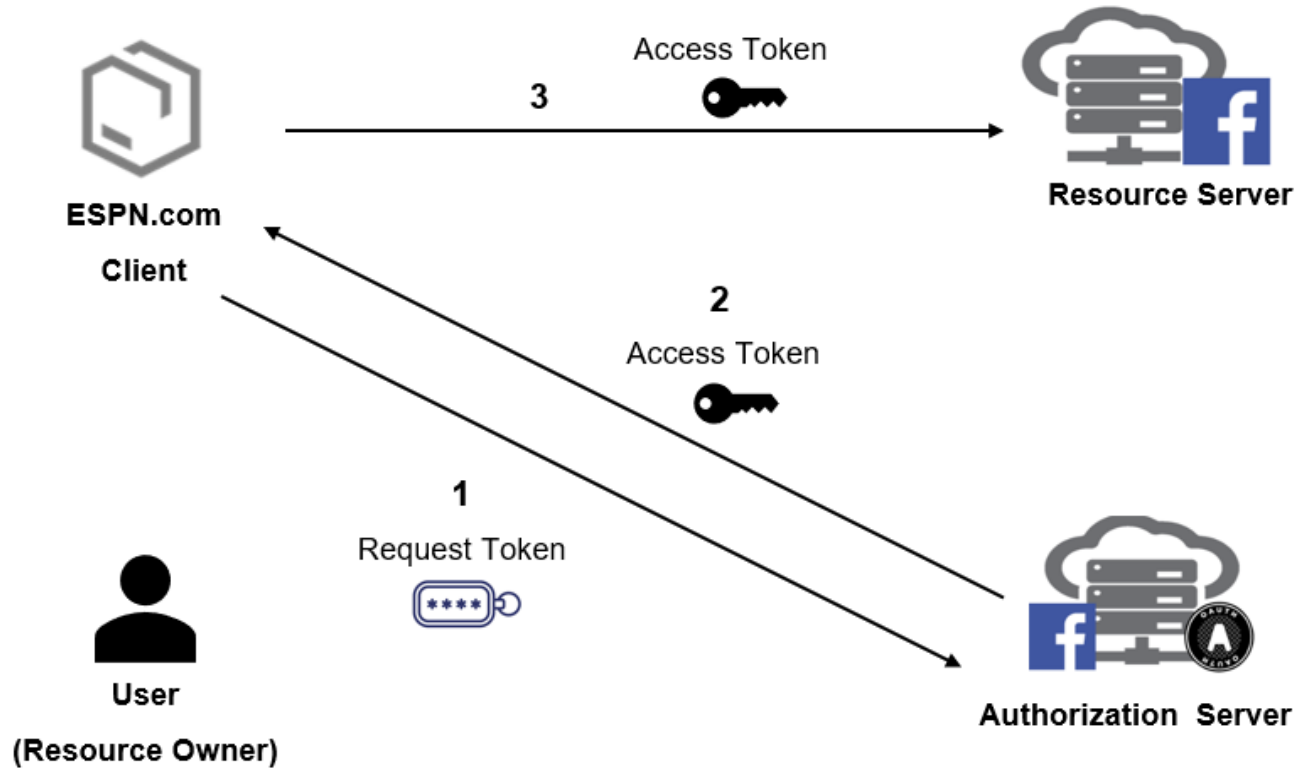
Authorization Server sends a consent dialog to User, saying "do you authorize ESPN to do X, Y and Z with your Facebook account?

Step 4 – User Confirms the Authorization Server request

User views the consent dialog and agrees.

A ***Request Token*** is sent to ESPN.

Back Channel Flow



Step 1 – ESPN asks for Access Token

ESPN: "Authorization Server, can I exchange this ***Request Token*** for an ***Access Token*** ()?"

Step 2 – Authorization Server provides Access Token

Authorization Server: "ESPN, here's your ***Access Token*** and Secret."

Step 3 – ESPN posts on User's Facebook Page

ESPN: "Facebook, I'd like to post this link to User's page. Here's my ***Access Token***"

Facebook: “Done!”

Let’s take another example:

You watch a video on YouTube, and you want to share that on your Facebook wall. Here:

- **You are the resource owner**
- **YouTube is the client**
- Facebook is the resource server and authorization sever.

OAuth 1.0 Vs OAuth 2.0

There are two versions of OAuth: OAuth 1.0a and OAuth 2.0. These specifications are completely different from one another and cannot be used together.

Nowadays, OAuth 2.0 is the most widely used form of OAuth. OAuth 2.0 is the one used in SAP BTP.

JWT Token

We learnt that OAuth call flows uses Access Token and Request Token. Usually, these tokens are JSON Web Tokens (JWT).

JWT (pronounced “jot”) is an **open standard** that defines a **compact** and **self-contained way** for **securely transmitting information** between parties.

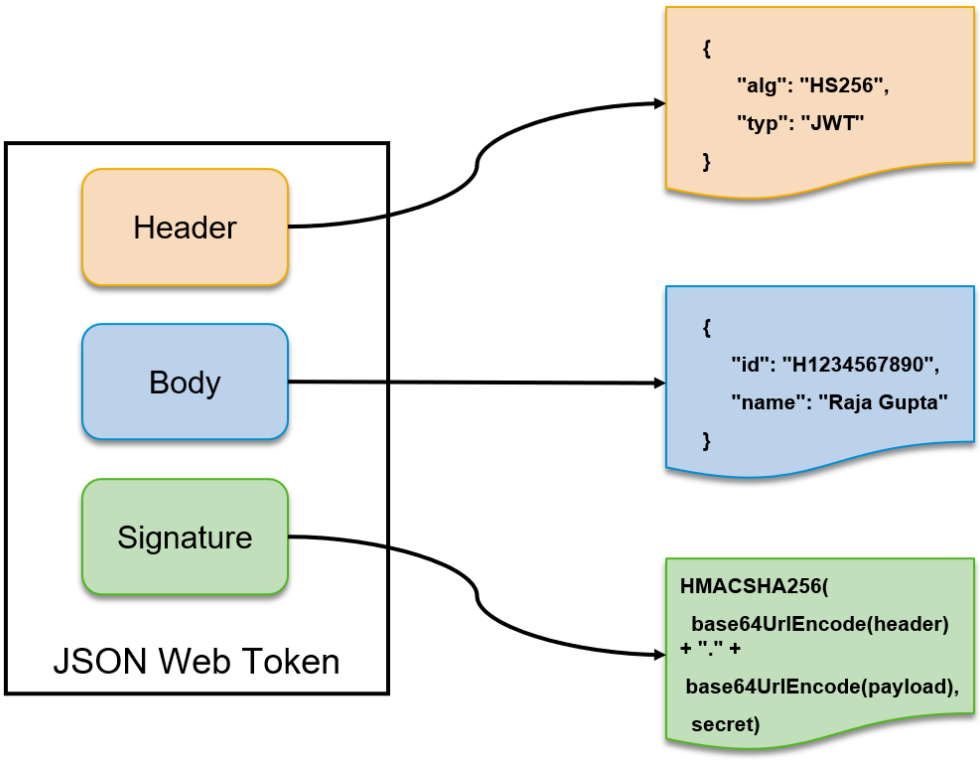
JWT is widely used in OAuth for securely transmit user information and access rights. You can check how XSUAA acts as the central administrator creating dedicated & tailored JWTs for our application and app router.

A JWT Token consists of three main components:

Header: Declaration of the used hashing and signing algorithms

Body/payload: Might be anything. For example, information about the user, the issuer and all the scopes.

Signature: The signature allows the integrity of the JWT.



All these parts can be combined and **converted with base64** so it is easier to add them in other data structures:

```
const token = base64urlEncoding(header)
              + "." + base64urlEncoding(payload)
              + "." + base64urlEncoding(signature)
```

Advantages of using JSON Web Token

JSON Web Tokens (JWT) are better than SAML tokens in many ways.

First of all, since JSON is less verbose than XML, JWT Tokens are smaller in size, making JWT more compact than SAML. This makes JWT a good choice to be passed over network.

JSON parsers are extremely common in most programming languages because they map directly to objects. Conversely, XML doesn't have a natural document-to-object mapping. This makes it easier to work with JWT than SAML assertions.

A simple real-life analogy of JWT token is – access card (or key) you use every day to enter your office. Like your access card has your user and access rights information, the JWT Token also contains the information about user e.g. id, email, address, access rights etc.

JWT token is cryptographically signed by the XSUAA, which means others cannot alter the user information of a token. However, if they somehow get access to the token, they can cause a lot of harm as they can access all the data the user has access to. That's the reason, JWT tokens are usually only be passed between applications and servers and never exposed to

end user. This is also the reason, why App Router takes care of attaching JWT token to request instead of sending JWT token to browser.

I hope you got the basic idea of OAuth and JWT Token.

Fundamentals of Security in BTP: Implement Authentication and Authorization in a Node.js App

This blog series is mainly targeted for developers and administrators. If you are someone who has gone through the plethora of tutorials, documentation, and presentations on security topics in SAP BTP and still lacks the confidence to implement security for your application, you have come to the right place.

In this blog series, you will learn:

- How to protect an app in SAP BTP, Cloud Foundry environment from unauthorized access
- How to implement role-based features
- How SAP Identity Provider and Single-sign-on works

For ease of reading, I have split this in multiple blogs, which are:

1. Fundamentals of Security in BTP: Introduction
2. Fundamentals of Security in BTP: What is OAuth? (optional)
3. Fundamentals of Security in BTP: Implement Authentication and Authorization in a Node.js App ***[current blog]***
4. Run Node.js Applications with Authentication Locally

Note: If you are new to SAP BTP and looking for a simple explanation of what it is and what problem it solves, see Explaining SAP Business Technology Platform (SAP BTP) to a Beginner

What are we going to learn in this blog?

So far, we have learnt about the core concepts of security. Now, let's get our hands dirty by implementing the authentication and authorization.

- We will first deploy an unsecured Node.js Hello World in BTP, Cloud Foundry
- Then we will implement authentication in the application
- Finally, we will add role based features for authorization

Note: Although, you can use any other IDE or command line interface to develop and deploy the Node.js app, we will recommend you use SAP Business Application Studio.

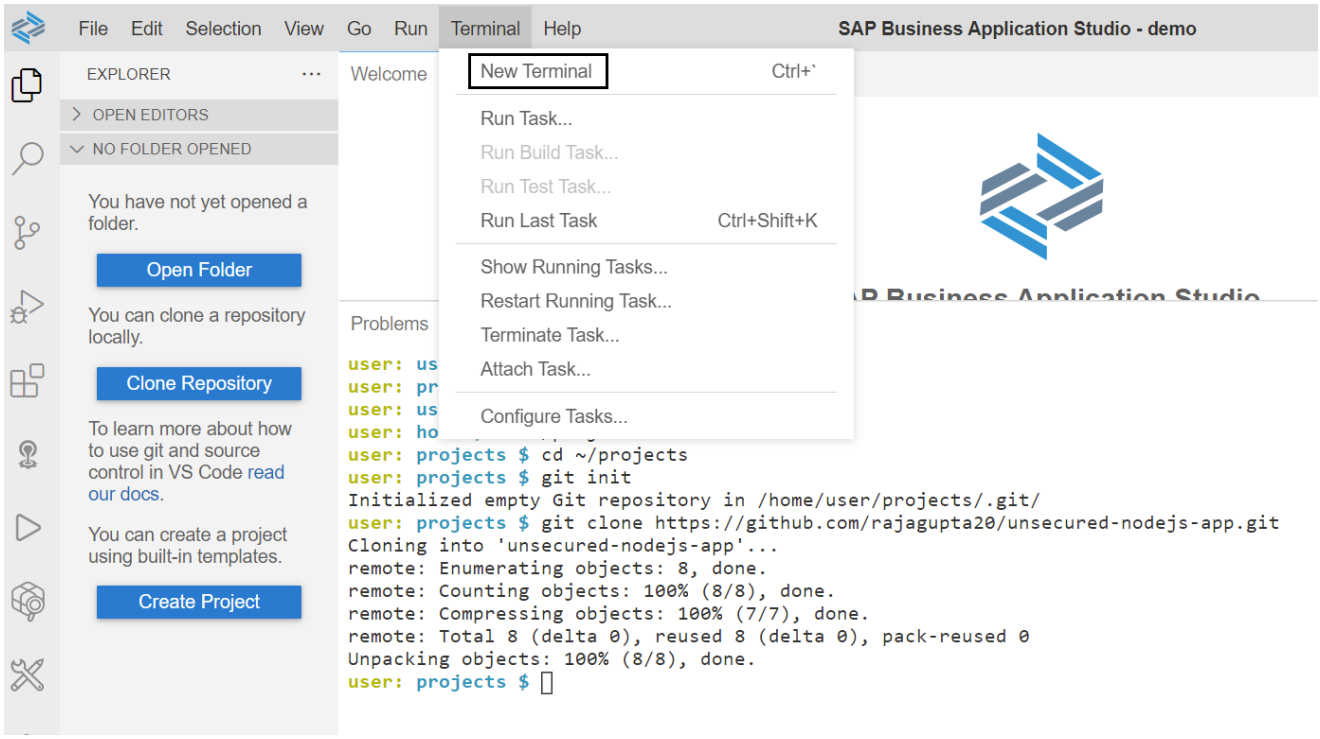
If you are new to SAP Business Application Studio, you may go through this tutorial to learn how to use it.

Let's first deploy an unsecured Node.js App

I have saved a simple Node.js hello world application in GitHub. Follow below steps to deploy it to Cloud Foundry environment of BTP.

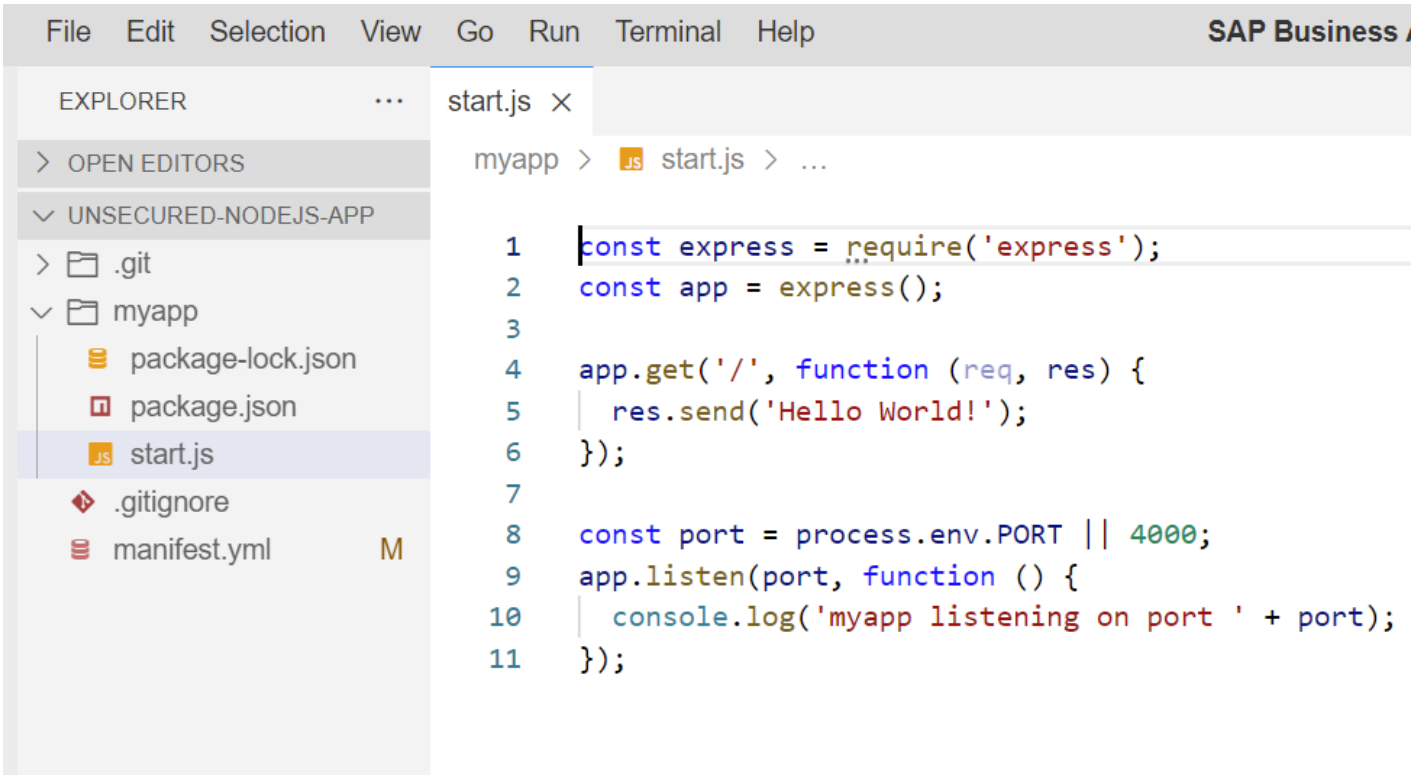
1. Open Business Application Studio. Go to ***Terminal*** -> ***New Terminal*** and run below command.

```
git init
git clone https://github.com/rajagupta20/unsecured-nodejs-app.git
```

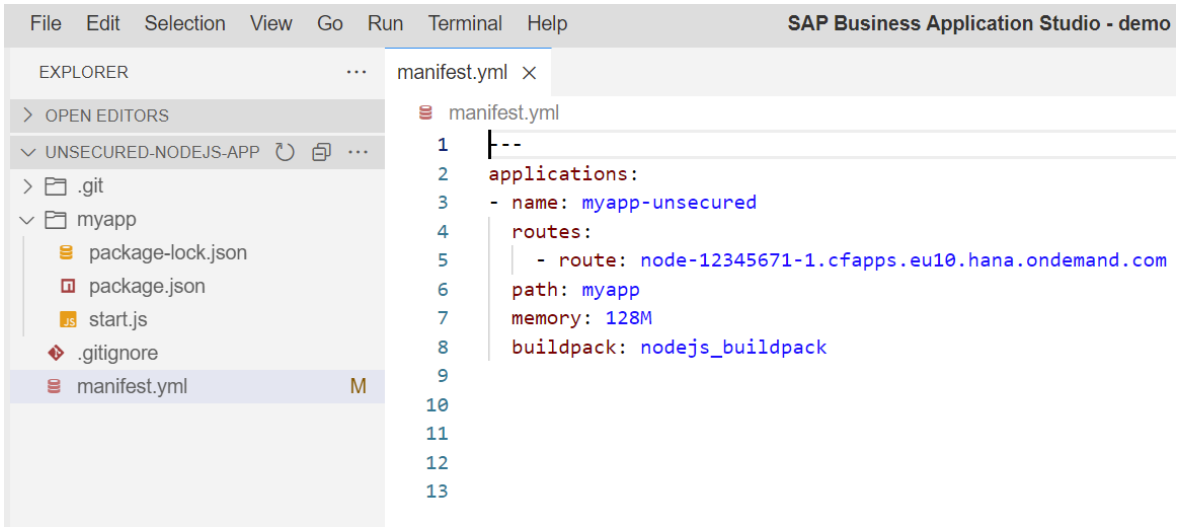


2. Click on ***Open Folder*** and select the application folder (unsecured-nodejs-app). Check below files.

start.js file – has basic hello world code



manifest.yml file – has runtime configurations



Execute the command ***cf push*** to deploy to your cloud foundry space and try to access the app.

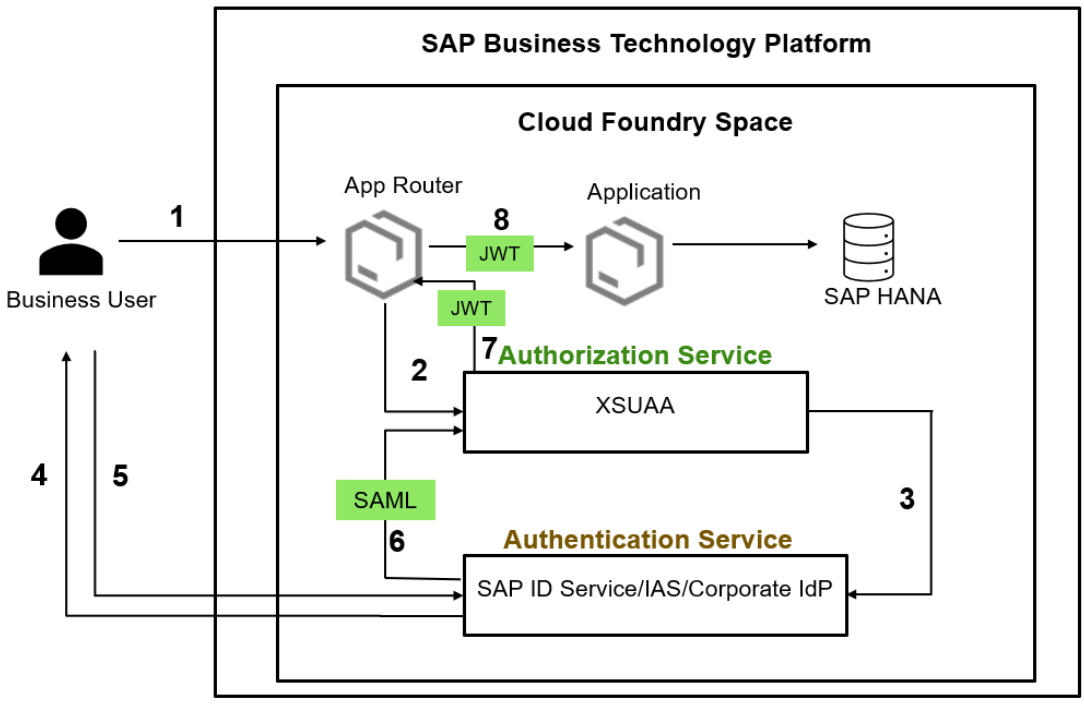
Result

You just deployed a Hello World Node.js application to BTP. You should be able to access the application without any authentication required.

Let's implement authentication to the Node.js App

Now, let's modify this Node.js application, so that only authenticated users will be able to access it.

Let's quickly recap what we learnt in previous blogs.



- A **business user** wants to access your application.
- The **single point of entry** is the **application router**.
- The **application router** sends the request to XSUAA.
- The XSUAA further **forwards** the request to Identity Provider**,** which takes care of authentication.

To implement authentication, all we need to do is:

1. Implement App Router
2. Create an instance of XSUAA service
3. Bind the application and App Router with XSUAA instance
4. Modify the application to make sure it only accepts request contains a JWT token

To make it simple, I have saved completed project in GitHub. Let's clone that by running below commands:

```
git init

git clone https://github.com/rajagupta20/secured-nodejs-app-demo1.git
```

Now, let's check authentication specific implementation.

1. Implement App Router

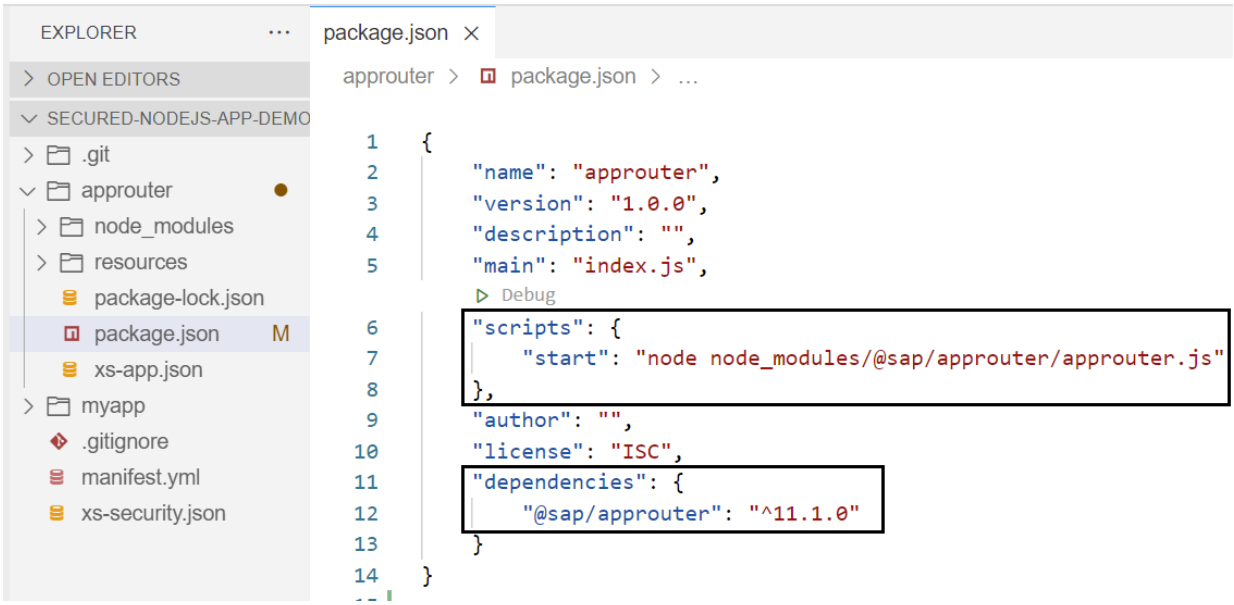
Remember – technically, Application Router is a Node.js App, available in public in NPM.

To add an App Router in our application, all we need to do is:

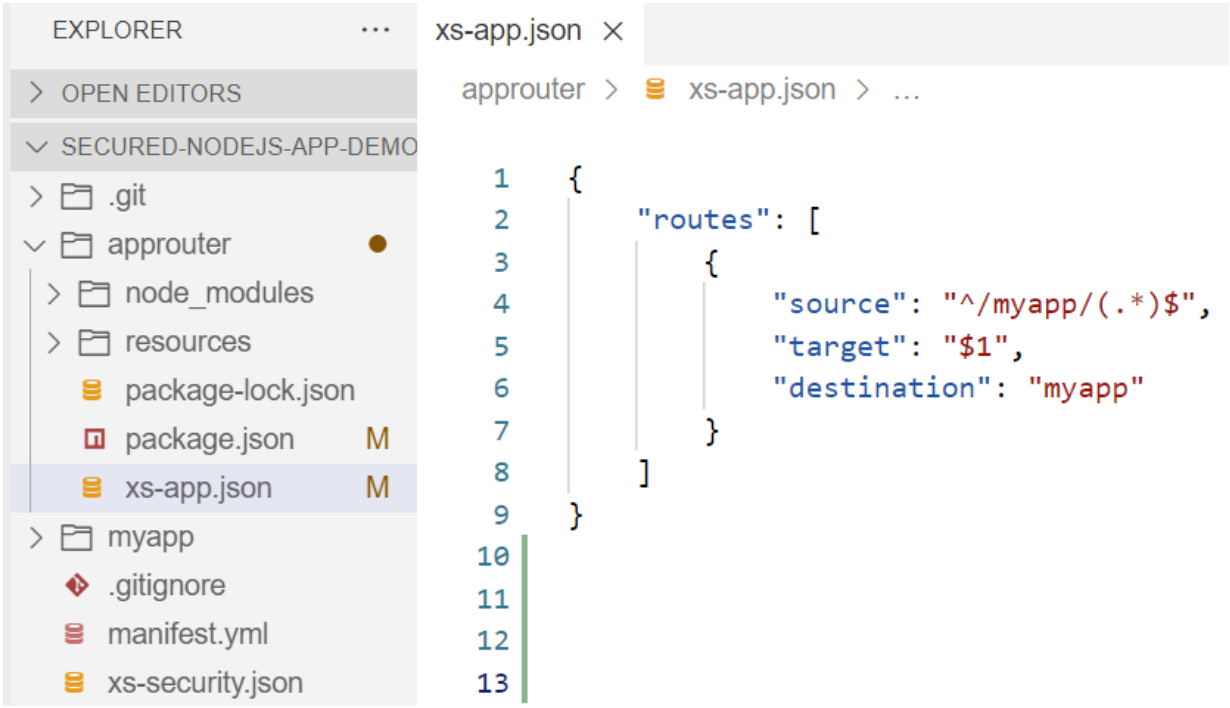
1. Create a folder (say approuter)
2. Create a ***package.json*** file inside the folder
3. In package.json file – **add the dependency for Approuter** and configure its start point
4. Create another file called ***xs-app.json*** inside approuter folder **to configure the App Router**

Check the cloned project and look into these 2 files.

package.json



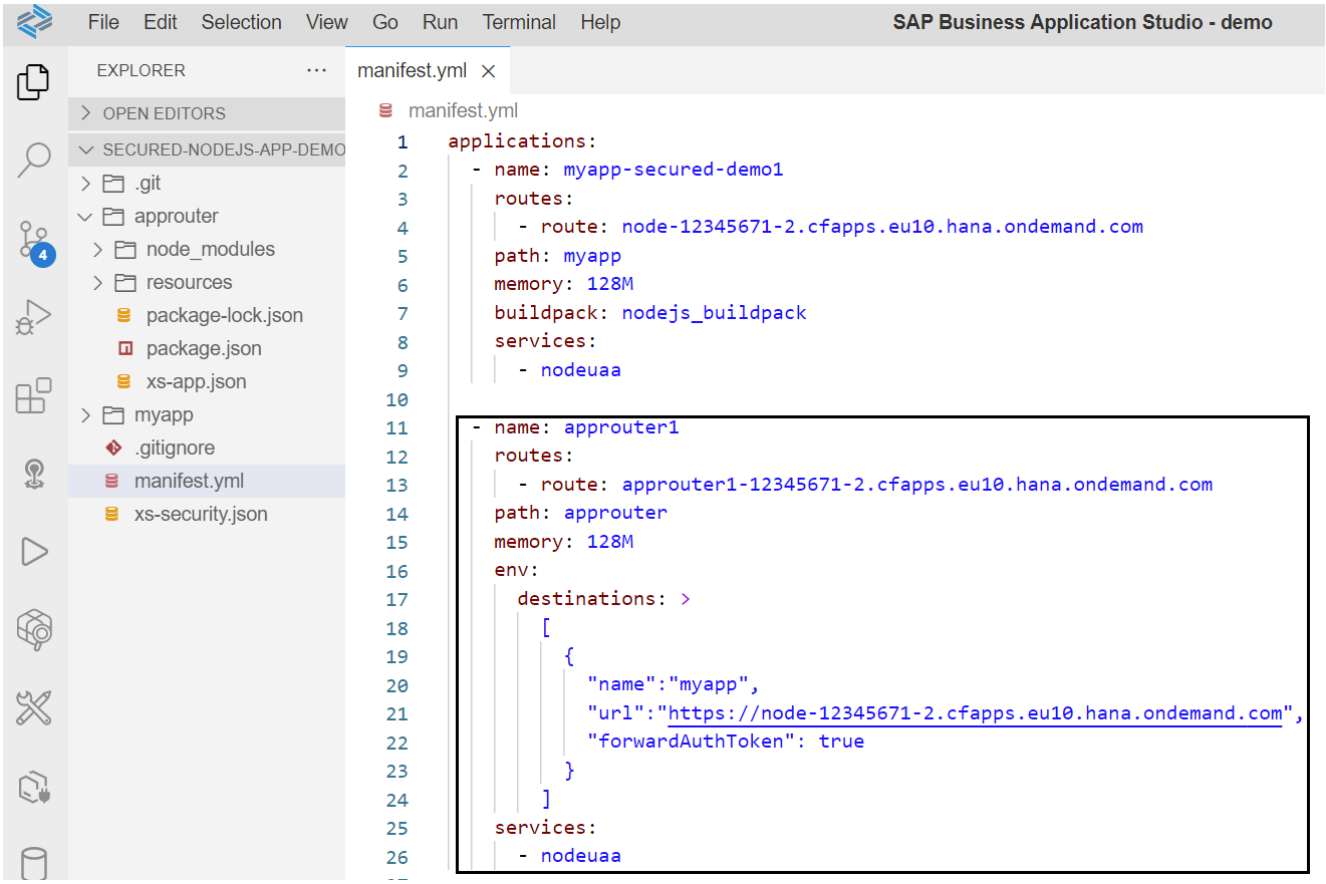
xs-app.json



xs-app.json (Routing Configuration File)

App Router's configuration is defined in the file called *xs-app.json*. In this example, we are keeping it simple by just adding a route. The route says – If the response URL pattern matches the given regex expression, forward it to a destination (here ***myapp***).

The destination ***(myapp)*** and App Router (***approuter1***) is specified in ***manifest.yml*** file as shown below.



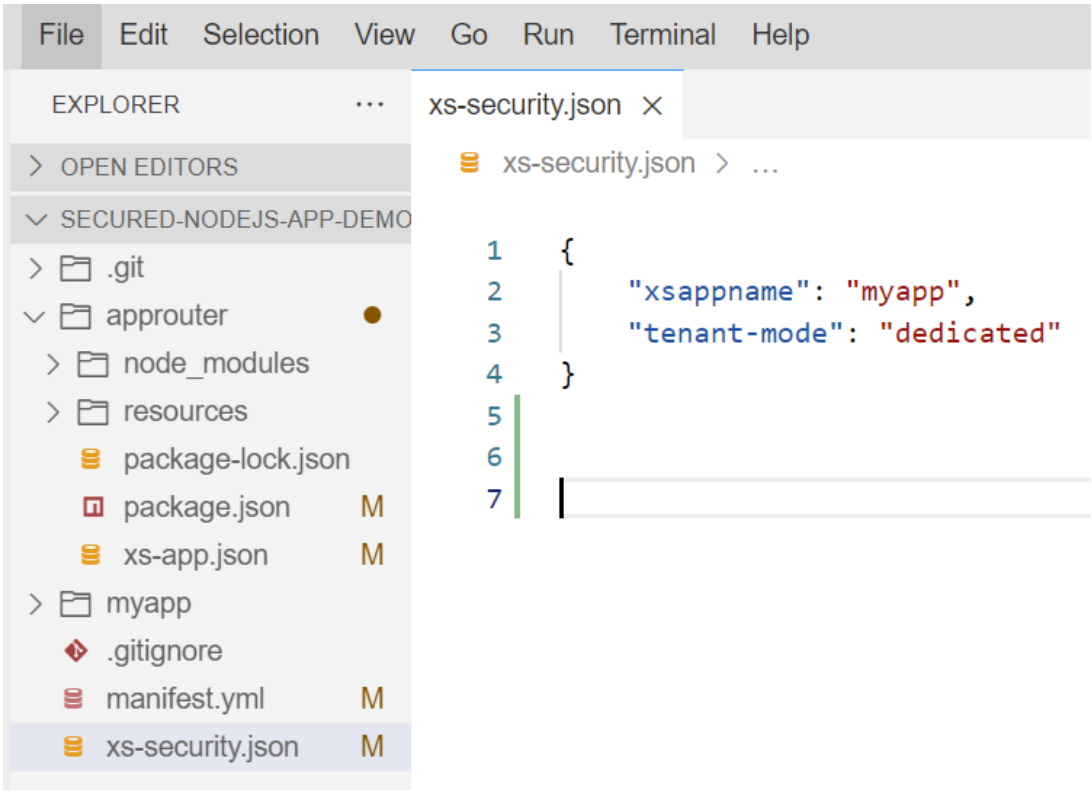
Note: In later blogs, we will look into *xs-app.json* file in detail.

2. Create an instance of XSUAA service

Before creating an instance of XSUAA, you need an important file called xs-security.json.

xs-security.json (Application Security Descriptor)

Check the file called xs-security.json in the cloned project.



xs-security.json is the file which defines the details of the **authentication methods** and **authorization types** to use for access to your application.

The *xs-security.json* file uses JSON notation to define the security options for an application. The **information in this file is used at application-deployment time**, for example, to create required roles for your application.

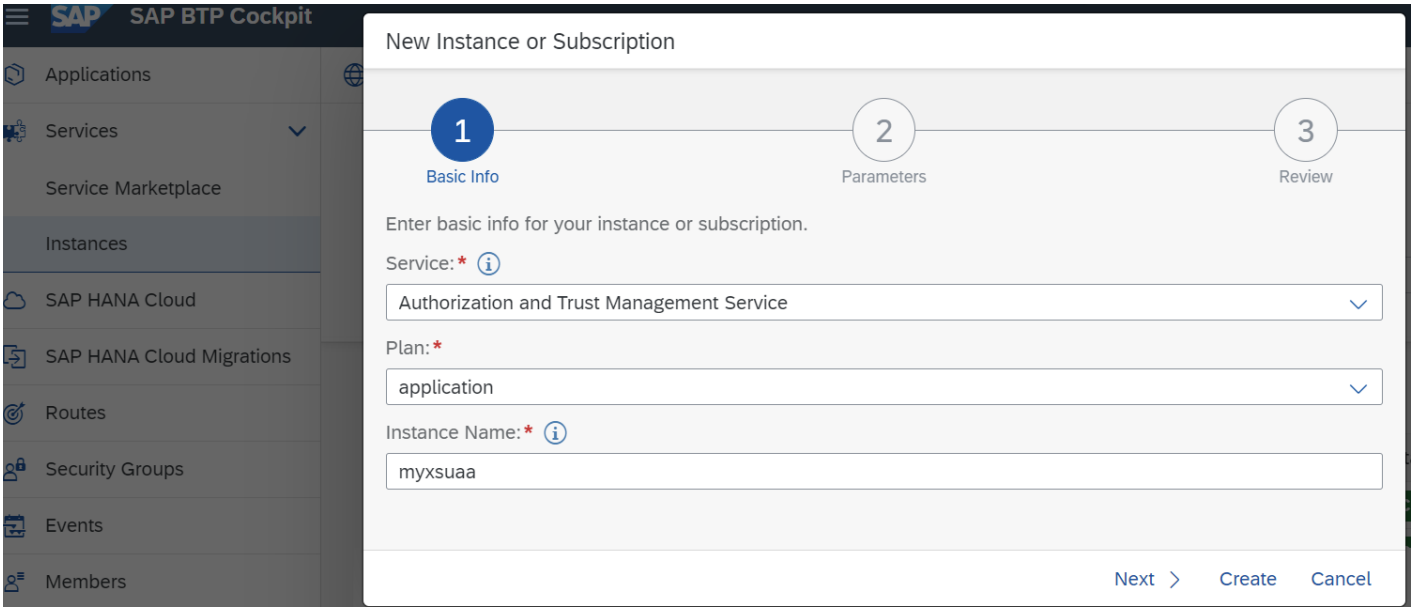
In this example, we are making it extremely simple, by just specifying xsappname and tenant-mode.

- **xsappname** property specifies the name of the application that the security description applies to.
- **tenant-mode** can be *"dedicated"* for single-tenant application and *"shared"* for multitenant application

Create instance of XSUAA using xs-security.json file

There are different ways to create an instance of XSUAA. Major options are:

1. Go to ***BTP Cockpit*** -> ***Subaccount*** -> ***Space*** -> ***Instance*** -> ***Create*** as shown below



2. Use command line tool. Execute below command

```
cf create-service xsuaa application <service_instance_name> -c xs-security.json
```

3. In case of Multitarget Applications (MTA), use configurations specified in ***yml*** file, which automatically creates XSUAA instances and bind it with applications. This is the most preferred way, and we will look into it later.

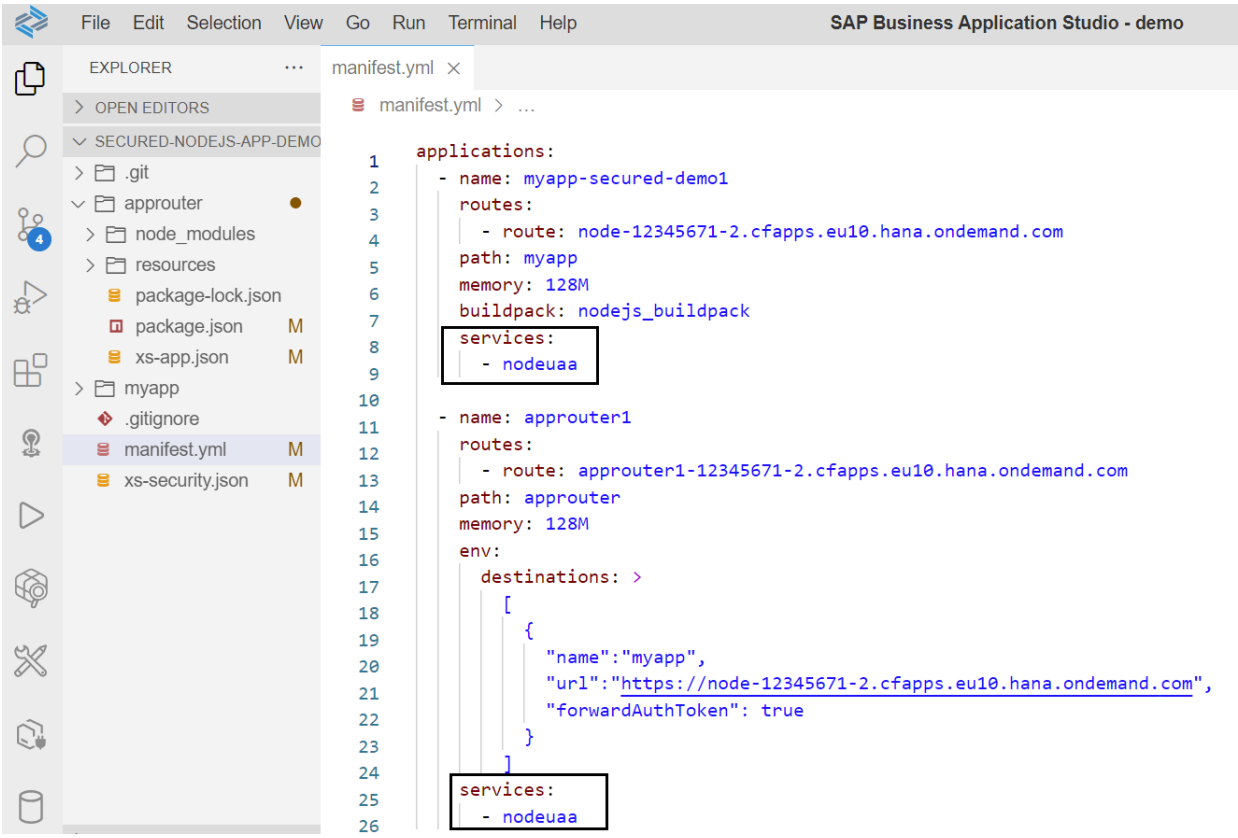
To make sure we understand what's happening behind the scenes, let's use command approach. Executing below command to create the XSUAA instance.

```
cf create-service xsuaa application nodeuaa -c xs-security.json
```

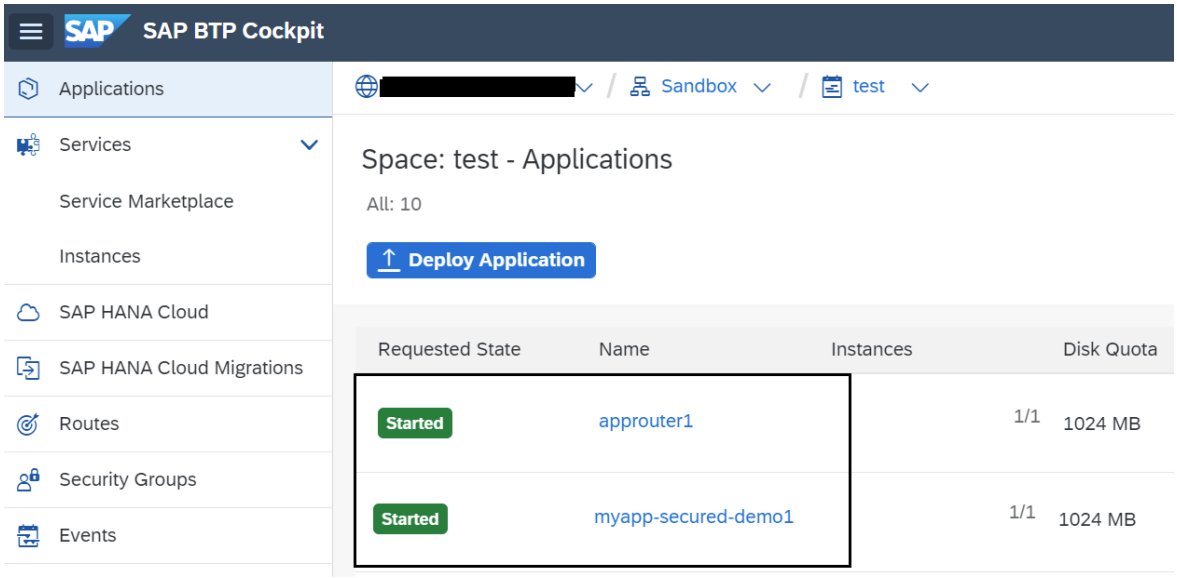
Here ***nodeuaa*** is the XSUAA instance name.

3. Bind the XSUAA instance with application and App Router

We can specify the binding of XSUAA instance with application and app router in *manifest.yml* file as shown below.

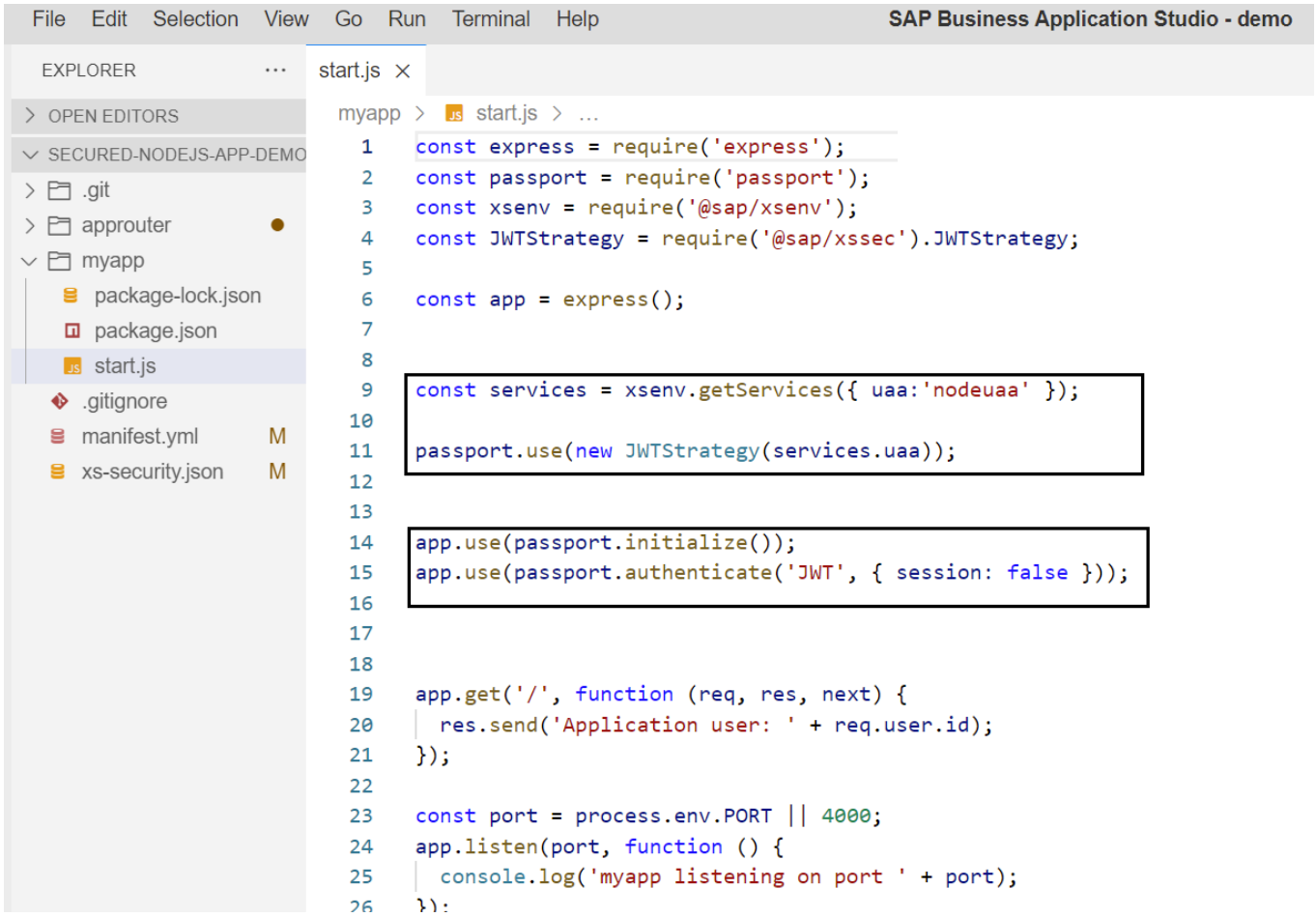


You can check the XSUAA instance and binding status in BTP cockpit as shown below.



4. Modify the application to make sure it only accepts request contains a JWT token

Finally, we need to make sure that application entertain only authenticated request containing a valid JWT token. In Node.js we can do that by using ***passport*** module.

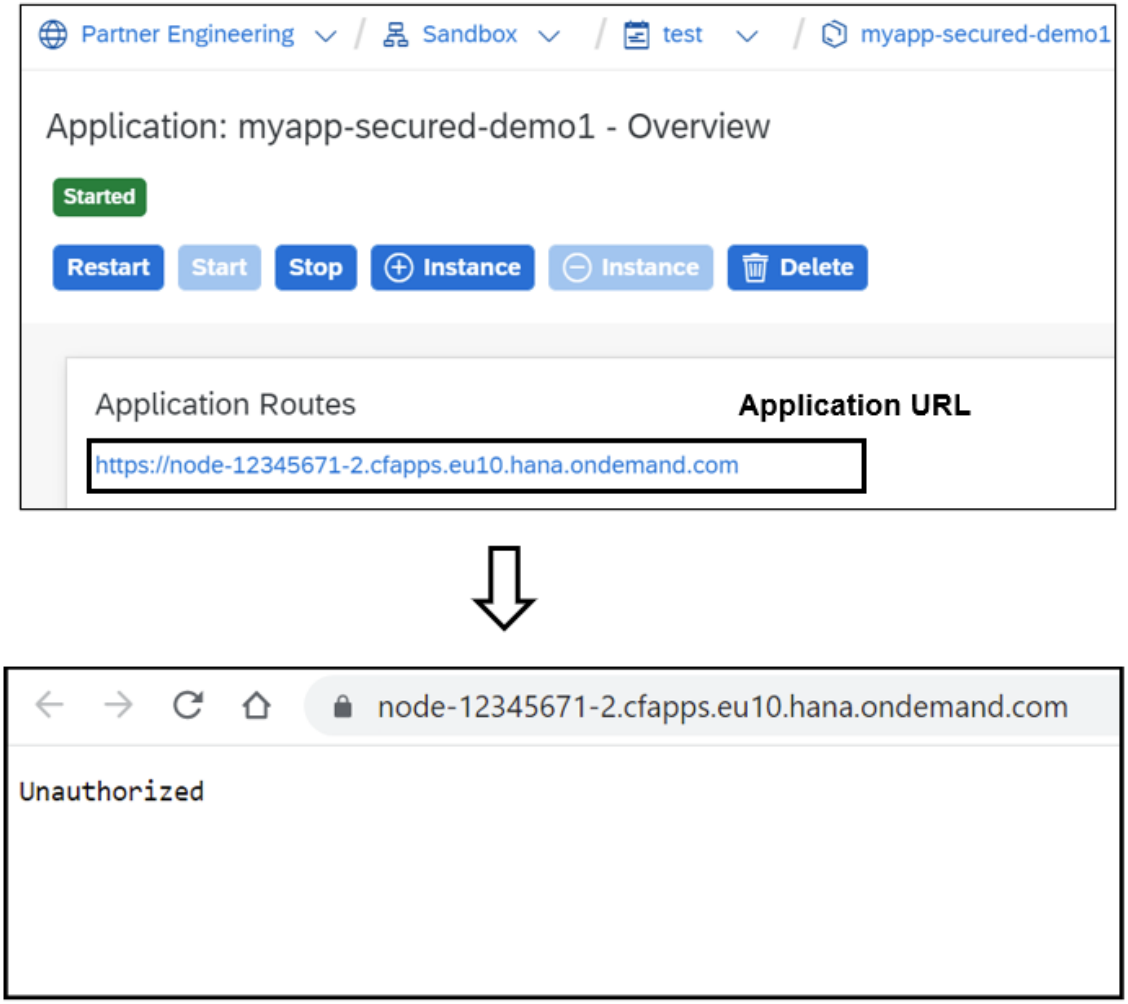


That's it. We have implemented the authentication in our Node.js application.

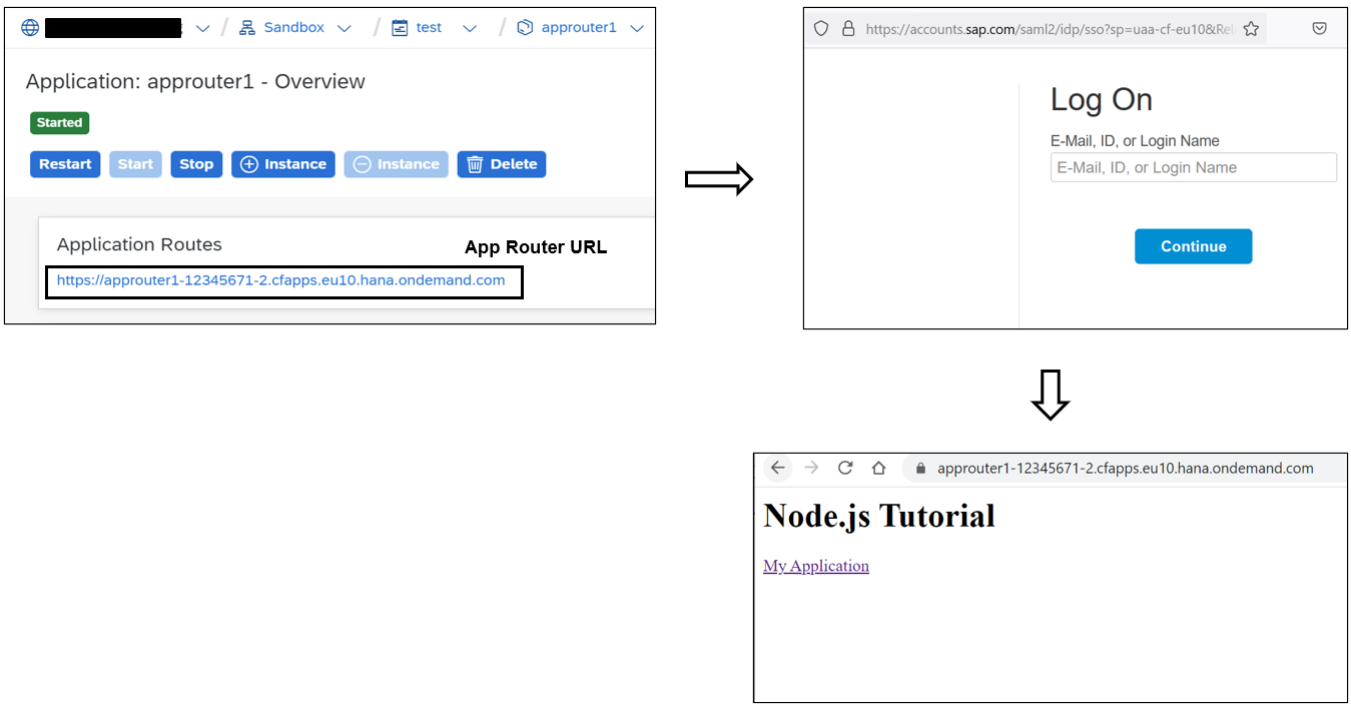
Execute ***cf push*** command to deploy the application. It will deploy both app router and application. You can check them in the BTP cockpit.

Test the Application

Try to access the application ("*myapp-secured-demo1*") directly. It would give "Unauthorized" error.



Now, try to **access the application via app router**, it will first **redirect to Identity Provider** and once authenticated (either using username/password or certificate-based login), it **redirects to application**. Below image shows the call flow if SAP ID Service is used as Identity Provider.



How Authorization works in SAP BTP, Cloud Foundry

Let’s move to authorization. Before doing the hands-on, let’s first understand few important points about XSUAA and user-role assignment.

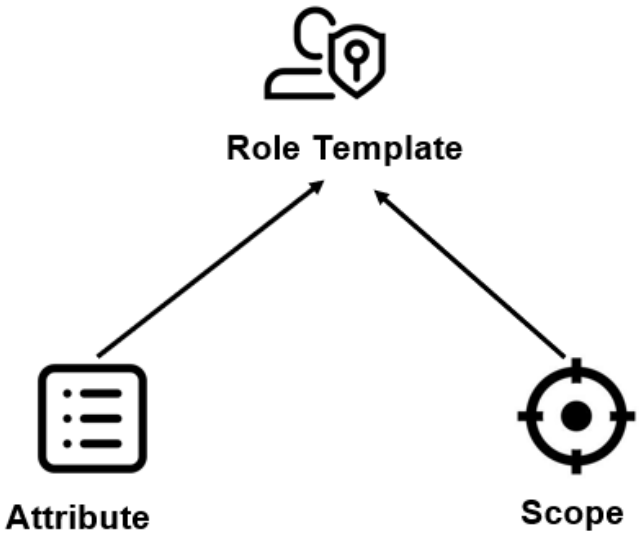
XSUAA Design Time and Runtime Artifacts

A quick recap:

- xs-security.json file is used to create an instance of XSUAA
- When XSUAA instance is being created, it generates few runtime artifacts

A typical xs-security.json file has 3 design time artifacts:

1. Scope
2. Attribute
3. Role-Collection



Here is a sample xs-security.json file.

```
{
    "xsappname": "myapp2",
    "tenant-mode": "dedicated",
    "scopes": [
        {
            "name": "$XSAPPNAME.Display",
            "description": "Display Users"
```

```
        },
        {
            "name": "$XSAPPNAME.Update",
            "description": "Update Users"
        }
    ],
    "role-templates": [
        {
            "name": "Viewer",
            "description": "View Users",
            "scope-references": [
                "$XSAPPNAME.Display"
            ]
        },
        {
            "name": "Manager",
            "description": "Maintain Users",
            "scope-references": [
                "$XSAPPNAME.Display",
                "$XSAPPNAME.Update"
            ]
        }
    ]
}
```

Scope

Scopes are functional authorizations that are assigned to users by means of security roles.

These scopes can be used for functional authorization checks. For example, in the application, we may check the scope using ***checkScope()*** function as shown below.

```
app.get('/users', function (req, res) {
    var isAuthorized =
            req.authInfo.checkScope('$XSAPPNAME.Display');
    if (isAuthorized) {
        res.status(200).json(users);
    } else {
        res.status(403).send('Forbidden');
    }
});
```

Attribute

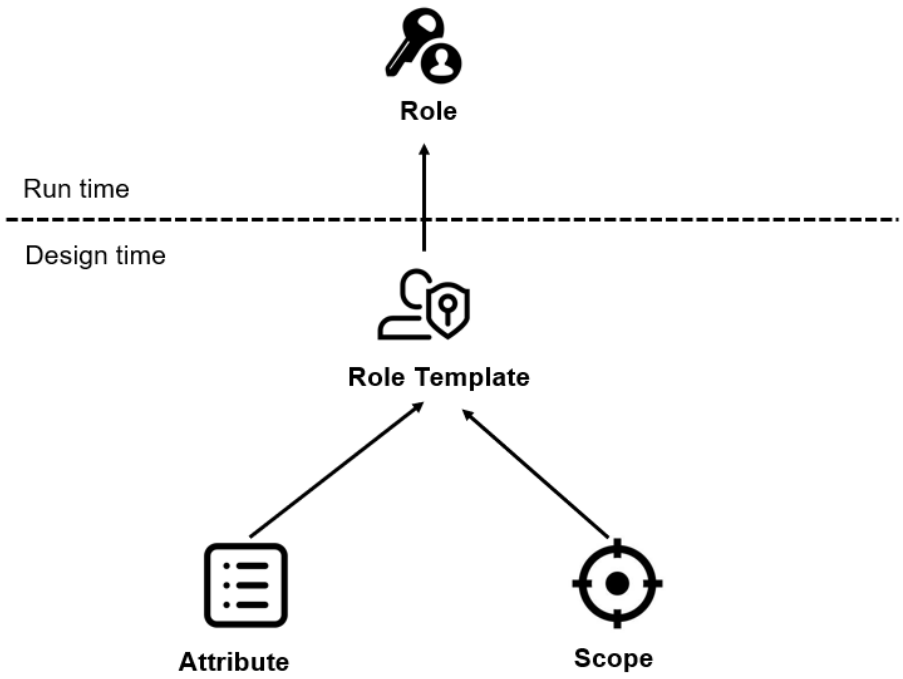
We can use attributes to perform more granular level checks. For example, a manager may have authorization to update employee data but **only for a specific country**.

In xs-security.json file we create the attribute (say country) and value of the country is assigned at runtime.

Role Templates

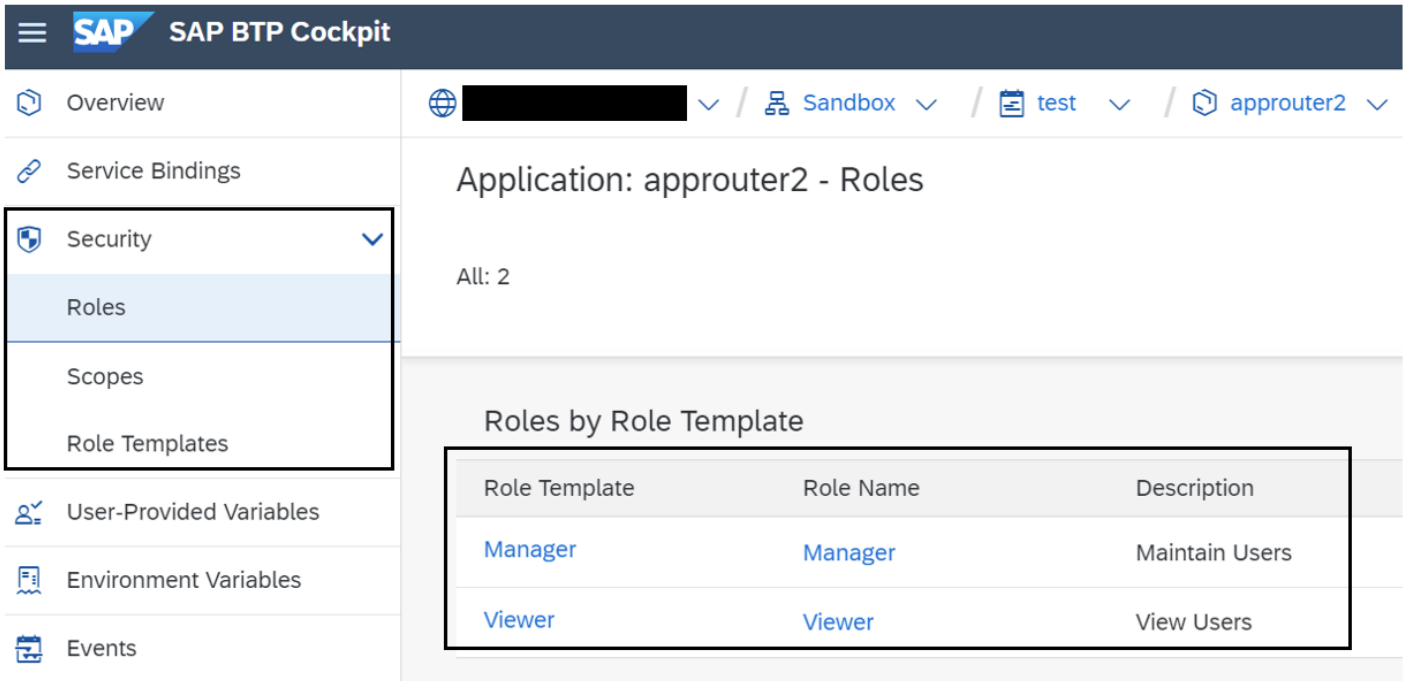
A role template combines the scopes and attributes. It is the description of roles (for example, “manager” or “employee”) to apply to a user.

When we create an XSUAA instance, it generates a runtime artifact called Role.



Role

Role is created based on Role Template at runtime. You can check the generated artefacts in the BTP Cockpit as shown below.

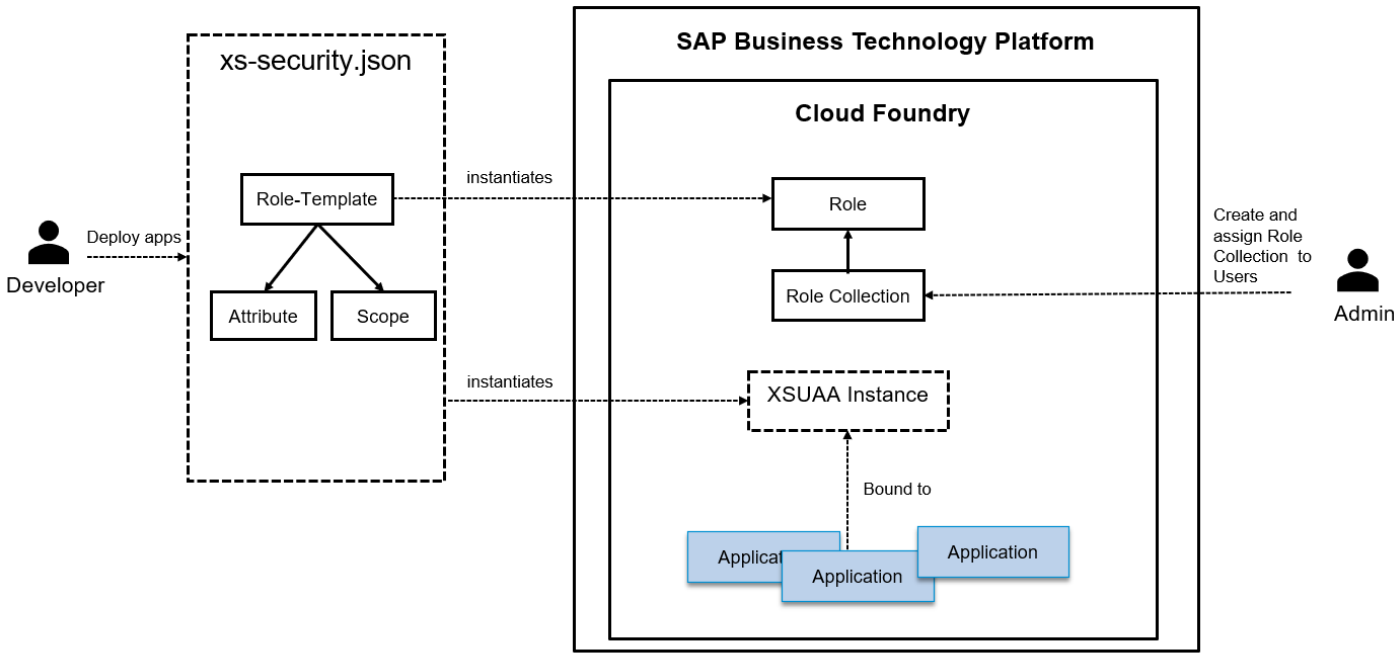


Assignment of Roles and Role Collection to Users

In BTP, there is something called **Role Collection** which helps to bind the Roles to Users.

Role Collection

Role Collections contain one or more roles. Further role collections are assigned to users by administrator.

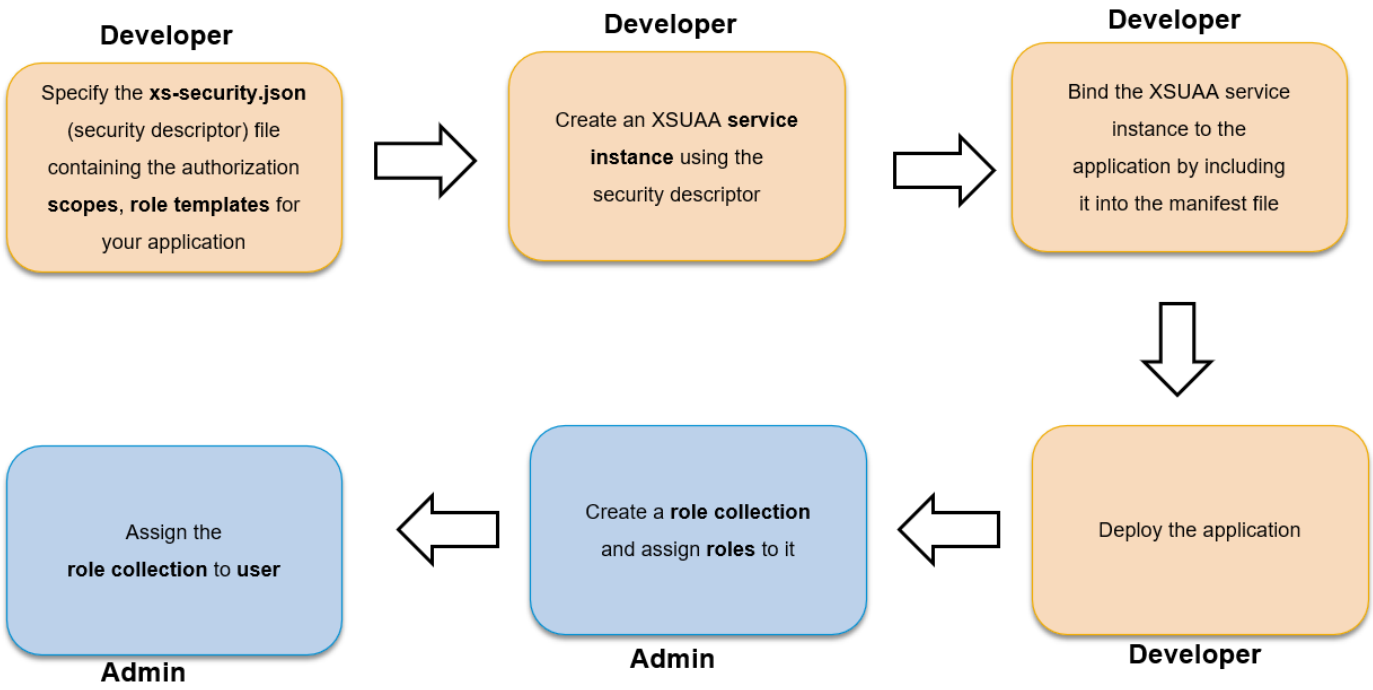


The above image shows:

- the design time (Role Template, Attribute & Scope) and run time artifacts (Role) of XSUAA. Role

- Roles are combined in Role Collection
- Role Collections is assigned to users
- XSUAA instance is also bound to the application

Below image shows the tasks/responsibilities of developer and administrator in this context.



Implement authorization in the Node.js application

Now, let's implement **authorization** to the same Node.js app and add some role-based features.

We will:

- Add 2 roles in the application – Manager and Viewer
- Implement that –
  - A user having **Viewer** role can only **view** the records
  - But a user having **Manager** role can **insert/delete** the records.

To make it simple, I have saved completed project in GitHub. Let's clone that by running below commands:

```
git init

git clone https://github.com/rajagupta20/secured-nodejs-app-demo2.git
```

Now, let's check authorization specific implementation.

1. Add Scope and Role Collection in xs-security.json file

We have added:

- 2 **Scopes** – Display and Update
- 2 **Role Templates** – Viewer and Manager

in xs-security.json file

```
{
    "xsappname": "myapp2",
    "tenant-mode": "dedicated",
    "scopes": [
        {
            "name": "$XSAPPNAME.Display",
            "description": "Display Users"
        },
        {
            "name": "$XSAPPNAME.Update",
            "description": "Update Users"
        }
    ],
    "role-templates": [
        {
            "name": "Viewer",
            "description": "View Users",
            "scope-references": [
                "$XSAPPNAME.Display"
            ]
        },
        {
            "name": "Manager",
            "description": "Maintain Users",
            "scope-references": [
                "$XSAPPNAME.Display",
                "$XSAPPNAME.Update"
            ]
        }
    ]
}
```

2. Implement role based features using Scope

In start.js file, notice the use of checkScope() function to implement role based feature. User can get the data only if he has *“Display”* scope. Similarly, user can edit the data only if he has *“Update”* scope.

```
const express = require('express');
const passport = require('passport');
const bodyParser = require('body-parser');
const xsenv = require('@sap/xsenv');
const JWTStrategy = require('@sap/xssec').JWTStrategy;

const users = require('./users.json');
const app = express();

const services = xsenv.getServices(
                            { uaa: 'nodeuaa2' }
                            );
passport.use(new JWTStrategy(services.uaa));

app.use(bodyParser.json());
app.use(passport.initialize());
app.use(passport.authenticate('JWT', { session: false }));

app.get('/users', function (req, res) {
    var isAuthorized =
            req.authInfo.checkScope('$XSAPPNAME.Display');
    if (isAuthorized) {
        res.status(200).json(users);
    } else {
        res.status(403).send('Forbidden');
    }
});


app.post('/users', function (req, res) {
    const isAuthorized = req.authInfo.checkScope('$XSAPPNAME.Update');
    if (!isAuthorized) {
        res.status(403).json('Forbidden');
        return;
    }

    var newUser = req.body;
    newUser.id = users.length;
    users.push(newUser);

    res.status(201).json(newUser);
});

const port = process.env.PORT || 4000;
app.listen(port, function () {
    console.log('myapp listening on port ' + port);
});
```

Note that we have a static resource ***approuter -> resources -> index.html***. There is **no authorization required** (no roles required) to access this file.

3. Create XSUAA instance and deploy the app

As we learned before, run below command to create XSUAA instance.

cf create-service xsuaa application nodeuaa2 -c xs-security.json

Note, that XSUAA instance name is nodeuaa2, the same is bound to application in *manifest.yml* file.

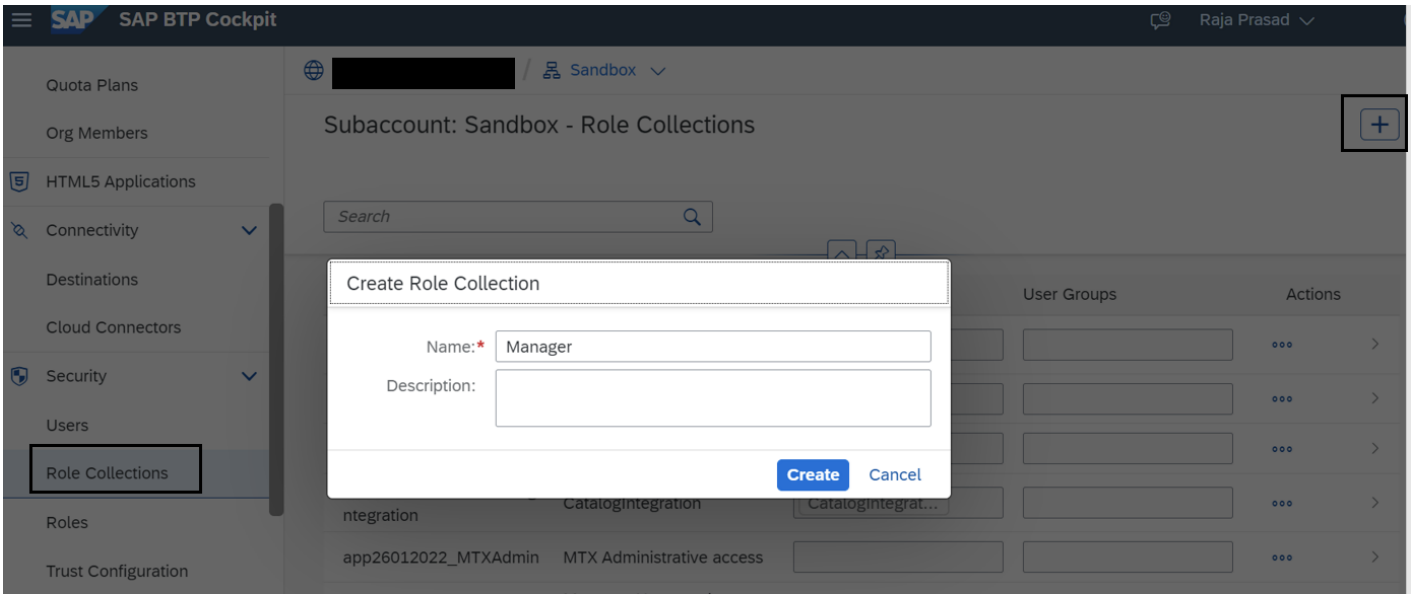
```
---
applications:
  - name: myapp-secured-demo2
    routes:
      - route: node-12345671-3.cfapps.eu10.hana.ondemand.com
    path: myapp
    memory: 128M
    buildpack: nodejs_buildpack
    services:
      - nodeuaa2

  - name: approuter2
    routes:
      - route: approuter1-12345671-3.cfapps.eu10.hana.ondemand.com
    path: approuter
    memory: 128M
    env:
      destinations: >
        [
          {
            "name":"myapp",
            "url":"https://node-12345671-3.cfapps.eu10.hana.ondemand.com",
            "forwardAuthToken": true
          }
        ]
    services:
      - nodeuaa2
```

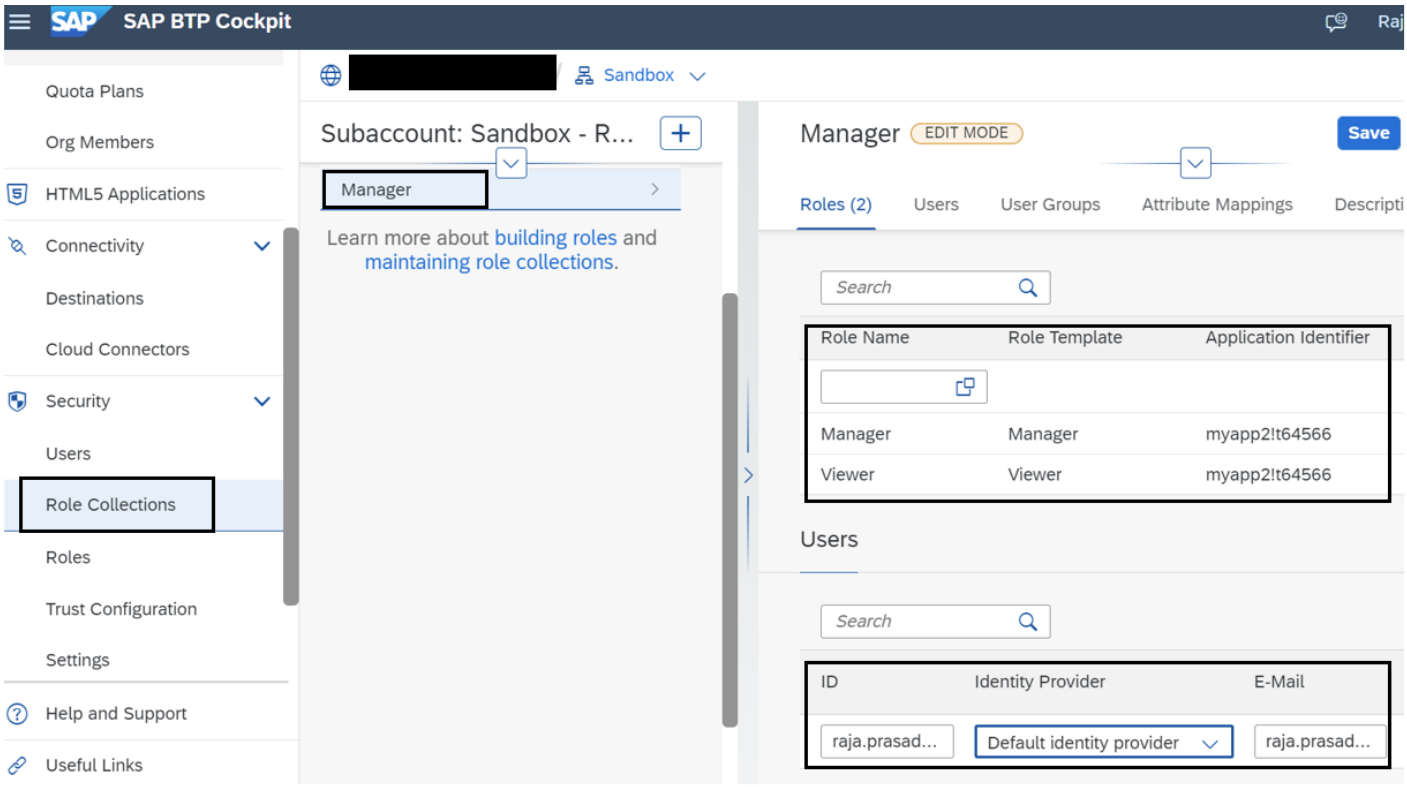
Next, execute ***cf push*** to deploy the app.

4. Create Role Collection and assign to user

Open BTP Cockpit and go to ***Subaccount -> Role Collections -> Create*** and create a role collection called Manager.



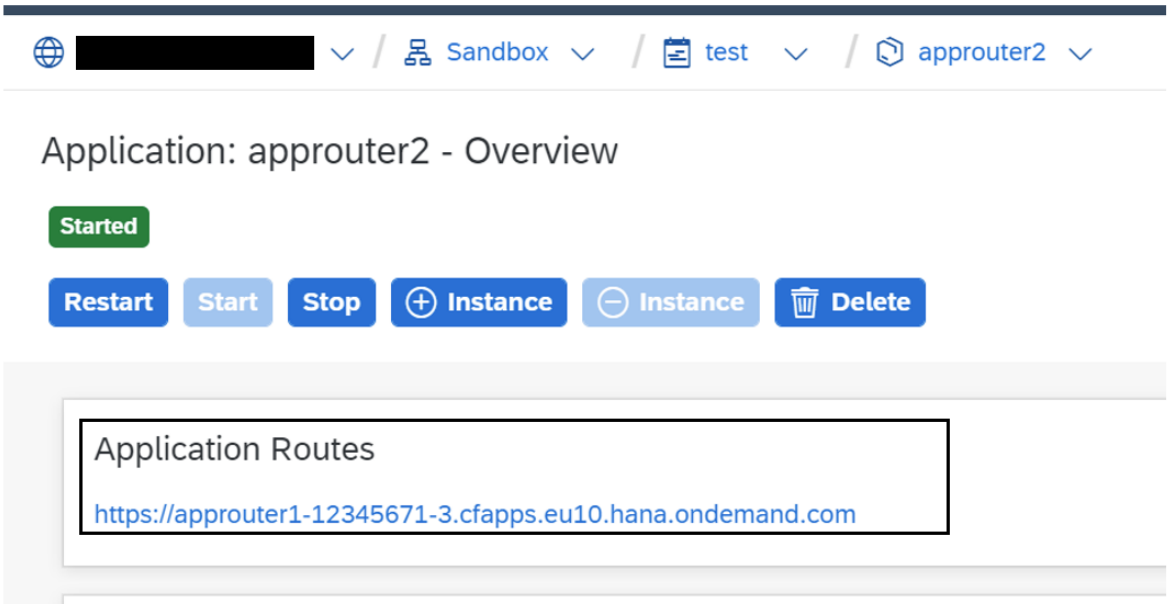
Select the role collection and add Manager and Viewer roles from your application. Also add your user (or any user you want to give access) to it.



Similarly, you may create another Role Collection called ***Viewer*** and only assign Viewer role to it.

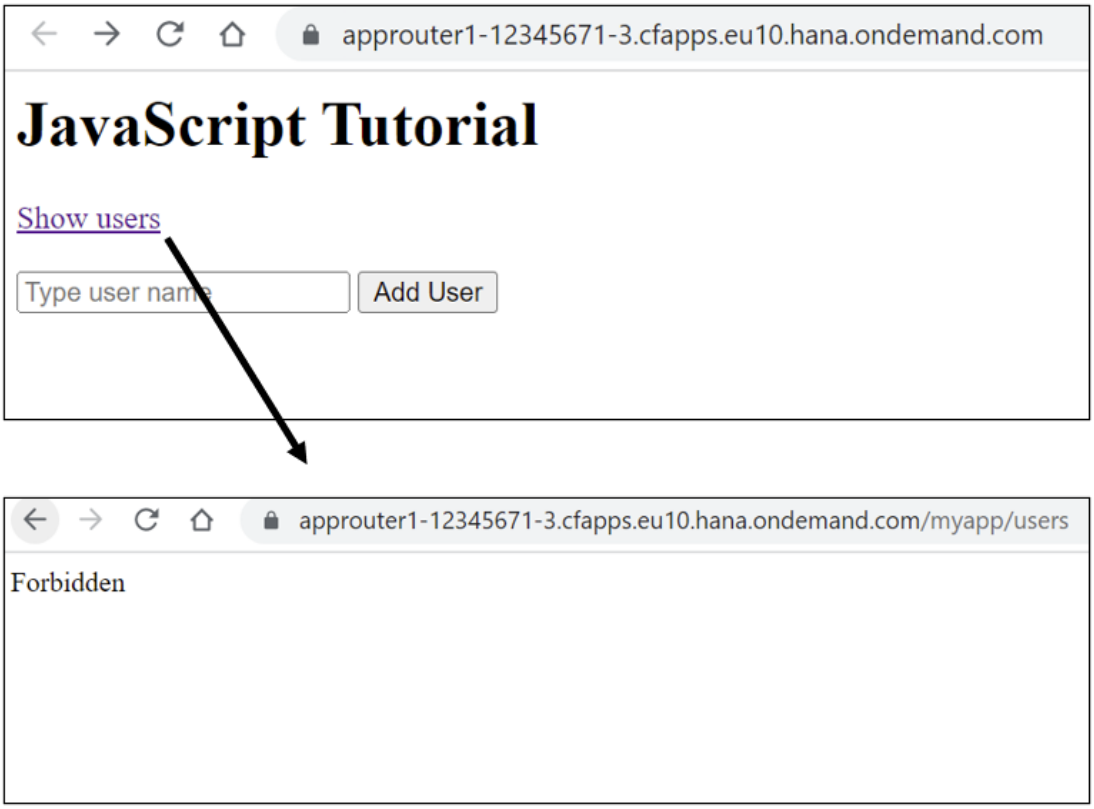
Test the Application

Now, try to access the application via App Router. You can get the App Router URL from cockpit as shown below.



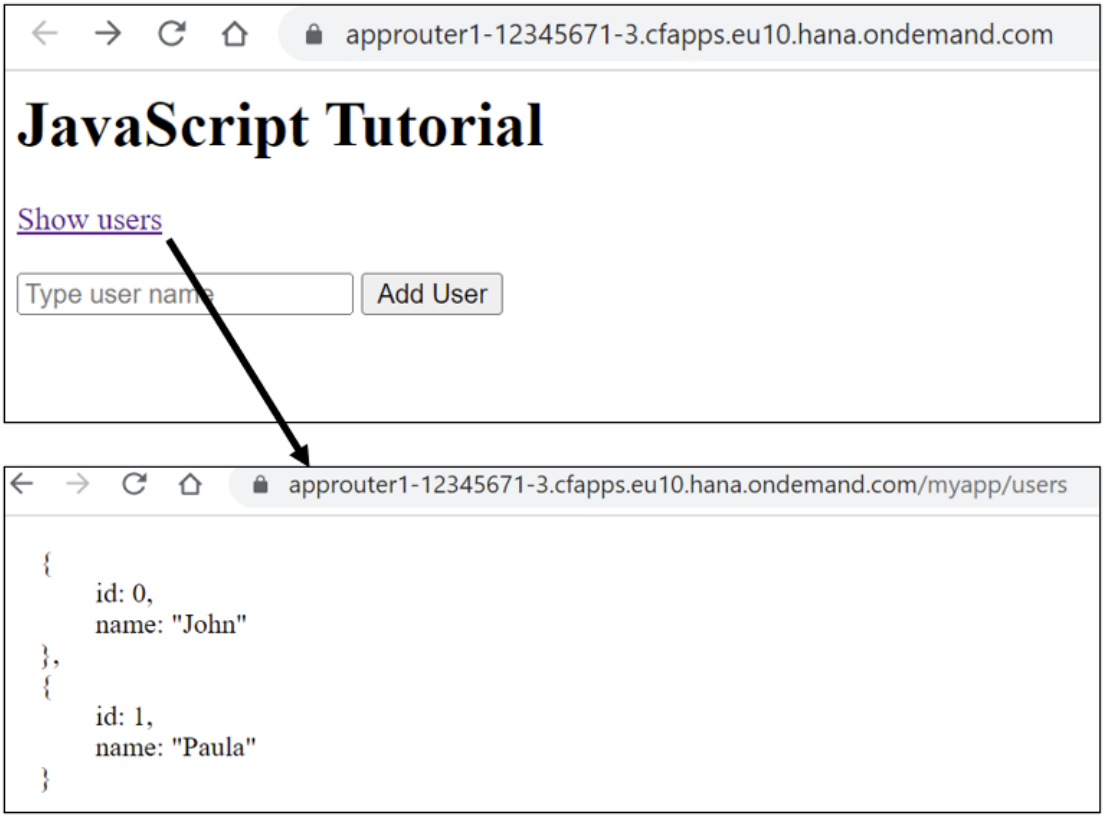
Scenario 1 – User does not have Manager or Viewer role collection assigned

You can access the static resource. But if you click on *"Show users"*, you will get *Forbidden* error message.

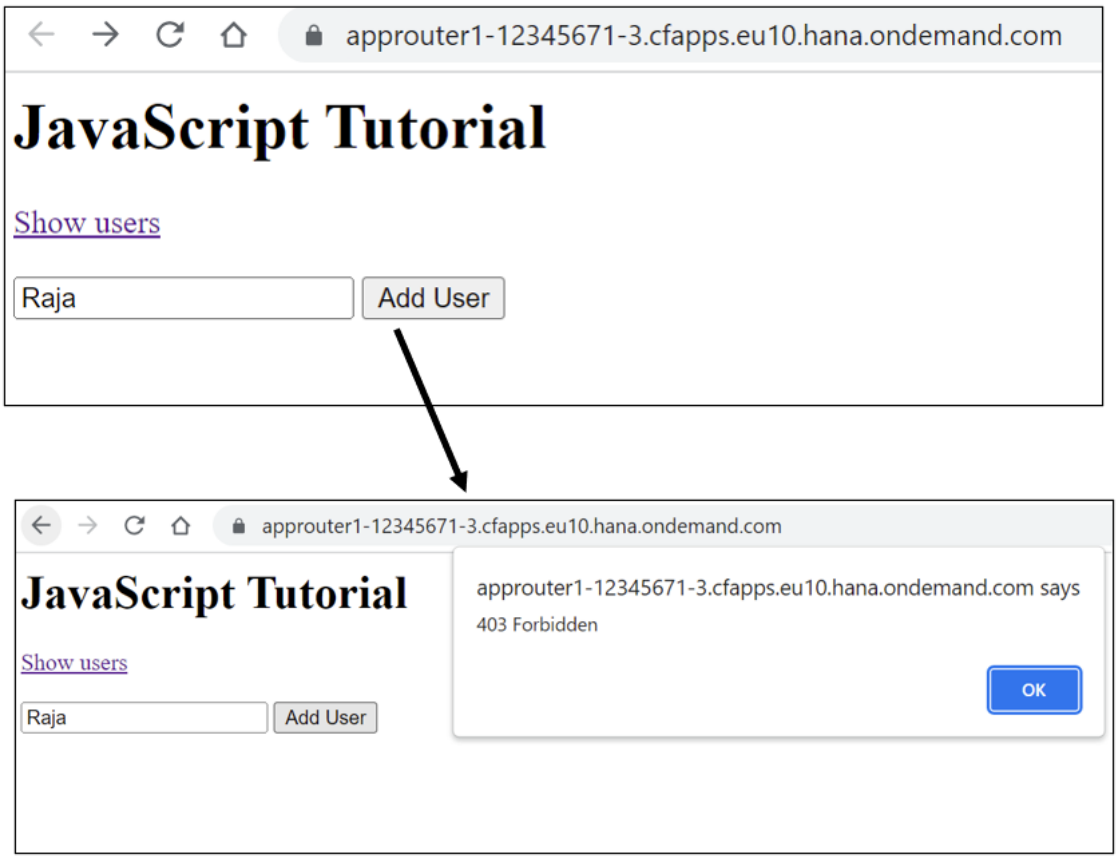


Scenario 2 – User has only Viewer role collection assigned

You can access the static resource as well as see the user data.



However, if you try to add a new user, you get ***forbidden*** error message.



Scenario 3 – User has Manager role collection assigned

You can access the static resource as well as see the user data. You can also add a new user.